BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI THÀNH PHỐ HỒ CHÍ MINH

BÁO CÁO THỰC TẬP TỐT NGHIỆP ĐẠI HỌC

***Đề tài*: " Xây dựng hệ thống nhận dạng khuôn mặt, giám sát an ninh, phân tích video thời gian thực"**

Người hướng dẫn	**:**	**Nguyễn Minh Tuấn**	
Sinh viên 1	**:**	**Trần Tuấn Hải**	**MSSV: N22DCCN026**
Lớp	**:**	**E22CQCNTT01-N**	
Sinh viên 2	**:**	**Huỳnh Bá Anh Khoa**	**MSSV: N22DCCN141**
Lớp	**:**	**E22CQCNTT01-N**	
Sinh viên 1	**:**	**Bùi Mạnh Khôi**	**MSSV: N22DCCN043**
Lớp	**:**	**E22CQCNTT01-N**	
Ngành	**:**	**Công nghệ thông tin**	

TP.HCM, tháng 08 /2026

MỤC LỤC

MỞ ĐẦU 12
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI 14
1.1 Vấn đề cấp thiết 14
1.2 Nhiệm vụ và mục tiêu đề tài 15
1.2.1 Mục tiêu tổng quát 15
1.2.2 Mục tiêu cụ thể 15
1.3 Đối tượng và phạm vi nghiên cứu 16
1.3.1 Đối tượng nghiên cứu 16
1.3.2 Phạm vi chức năng 17
1.3.3 Phạm vi dữ liệu và môi trường 18
1.3.4 Phạm vi của báo cáo hiện tại 18
1.4 Phương pháp nghiên cứu và quy trình thực hiện 19
1.4.1 Phương pháp nghiên cứu 19
1.4.2 Quy trình thực hiện 21
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG 23
2.1 Tổng quan về trí tuệ nhân tạo, học máy và học sâu 23
2.2 Xử lý ảnh, phát hiện người và khuôn mặt 24
2.2.1 Tiền xử lý và kiểm tra chất lượng ảnh 24
2.2.2 Phát hiện khuôn mặt bằng InsightFace và SCRFD 25
2.2.3 Căn chỉnh và cắt khuôn mặt 25
2.2.4 Phát hiện người bằng YOLOv8 26
2.3 Trích xuất và so khớp đặc trưng khuôn mặt 27
2.3.1 Biểu diễn khuôn mặt bằng embedding 27
2.3.2 Nguyên lý ArcFace 27
2.3.3 So khớp bằng cosine similarity 28
2.4 Theo dõi đối tượng và phân tích hành vi 28
2.4.1 Theo dõi đa đối tượng bằng ByteTrack 28
2.4.2 Phân tích hành vi dựa trên quy tắc 29
2.5 Công nghệ phát triển hệ thống 29
2.5.1 Backend FastAPI 29
2.5.2 Giao diện Flutter 31
2.5.3 Cơ sở dữ liệu SQLite 31
2.6 Các chỉ số đánh giá hệ thống 32

2.6.1 Chỉ số đánh giá nhận dạng....32
2.6.2 Chỉ số đánh giá phát hiện và theo dõi....33
2.6.3 Chỉ số hiệu năng....33
2.7. Kết luận chương....34
CHƯƠNG 3: PHÂN TÍCH YÊU CẦU HỆ THỐNG....35
3.1 Yêu cầu chức năng....35
3.1.1 Tiếp nhận và xử lý luồng video thời gian thực....35
3.1.2 Đăng ký thông tin và lưu trữ thư viện khuôn mặt (Enrollment)....35
3.1.3 Nhận dạng danh tính và phát hiện người lạ....35
3.1.4 Thiết lập vùng cấm và cảnh báo hành vi bất thường....36
3.1.5 Hiển thị và xử lý cảnh báo trên ứng dụng....36
3.2 Yêu cầu phi chức năng....36
3.2.1 Hiệu năng và tốc độ xử lý....36
3.2.2 Độ tin cậy và tỷ lệ cảnh báo sai (False Alarm Rate)....37
3.3 Kịch bản sử dụng hệ thống (Use Case Diagram & Đặc tả chi tiết)....38
3.3.1 Danh sách Actor....38
3.3.2 Sơ đồ Use Case tổng quan....38
3.3.3 Đặc tả chi tiết từng Use Case....39
3.4 Đối chiếu yêu cầu với phạm vi triển khai hiện tại....48
CHƯƠNG 4: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG....49
4.1 Kiến trúc mã nguồn sản phẩm....49
4.2 Thiết kế cơ sở dữ liệu quan hệ....50
4.3 Ổn định Backend và quản lý cấu hình....53
4.4 Cơ sở dữ liệu và ranh giới lưu trữ....53
4.5 Bộ máy AI dùng chung....55
4.6 Quản lý camera và giám sát webcam....55
4.7 Nhật ký và cảnh báo webcam....56
4.8 Phân tích video tải lên tạm thời....57
4.9 Tích hợp ứng dụng Flutter....57
CHƯƠNG 5: ĐÁNH GIÁ, SO SÁNH GIỮA CÁC MODELS....58
5.1 So sánh các model phát hiện khuôn mặt....58
5.2 So sánh các model nhận diện khuôn mặt....60
5.3 So sánh các models theo vết người....62
CHƯƠNG 6: KẾT QUẢ TRIỂN KHAI VÀ KIỂM THỬ....65

6.1 Chiến lược kiểm thử 65
6.2 Kết quả kiểm thử tự động 65
6.3 Các kịch bản quan trọng đã kiểm chứng 66
6.4 Kết quả chức năng theo hai chế độ 67
6.5 Khả năng vận hành và chẩn đoán 68
CHƯƠNG 7: ĐÁNH GIÁ VÀ KẾ HOẠCH TIẾP THEO 70
7.1 Mức độ hoàn thành theo phân công 70
7.2 Hạn chế hiện tại 71
7.3 Kế hoạch giai đoạn tiếp theo 72
KẾT LUẬN 73
TÀI LIỆU THAM KHẢO 75

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc đến thầy Nguyễn Minh Tuấn, người đã trực tiếp hướng dẫn, định hướng và tận tình hỗ trợ chúng em trong suốt quá trình thực hiện dự án thực tập tốt nghiệp. Những ý kiến đóng góp, sự hướng dẫn và kinh nghiệm thực tiễn của thầy đã giúp chúng em hoàn thành dự án cũng như bài báo cáo này.

Chúng em xin chân thành cảm ơn quý thầy, cô của Học viện Công nghệ Bưu chính Viễn thông đã tận tình giảng dạy, truyền đạt những kiến thức chuyên môn và kỹ năng cần thiết trong suốt quá trình học tập. Đây là nền tảng quan trọng giúp chúng em có thể vận dụng vào quá trình thực hiện dự án thực tập tốt nghiệp.

Bên cạnh đó, chúng em cũng xin gửi lời cảm ơn đến các anh chị, đồng nghiệp và các bạn đã luôn sẵn sàng chia sẻ kinh nghiệm, đóng góp ý kiến và hỗ trợ chúng em trong quá trình thực hiện dự án.

Mặc dù đã nỗ lực hết sức để hoàn thành dự án và báo cáo, do kiến thức và kinh nghiệm thực tế còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ thầy Nguyễn Minh Tuấn để có thể tiếp tục hoàn thiện kiến thức và nâng cao kỹ năng trong thời gian tới.

Cuối cùng, chúng em xin gửi lời cảm ơn chân thành đến gia đình, người thân và bạn bè đã luôn động viên, khích lệ và tạo mọi điều kiện thuận lợi để chúng em hoàn thành tốt kỳ thực tập cũng như bài báo cáo này.

Thành phố Hồ Chí Minh, tháng 8 năm 2026

Sinh viên thực hiện

Trần Tuấn Hải

Huỳnh Bá Anh Khoa

Bùi Mạnh Khôi

DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH

Bảng 3.1 Bảng use case đăng ký danh tính 40
Bảng 3.2 Bảng use case quản lý camera 41
Bảng 3.3 bảng use case cấu hình vùng cấm 42
Bảng 3.4 Bảng use case cấu hình luật hành vi 43
Bảng 3.5 Bảng use case thu nhận luồng video 44
Bảng 3.6 Bảng use case phát hiện người và khuôn mặt 44
Bảng 3.7 Bảng use case nhận diện danh tính 45
Bảng 3.8 Bảng use case theo dõi đối tượng 46
Bảng 3.9 Bảng use case đánh giá luật hành vi & sinh ra cảnh báo 47
Bảng 3.10 Bảng use case đẩy thông báo 48
Bảng 3.11 Bảng use case xem và xử lý danh sách cảnh báo 48
Bảng 3.12 Bảng use case quản lý tài khoản và thiết bị 49
Bảng 4.1 Các lớp chức năng của hệ thống 51
Bảng 4.2 Lưu trữ dữ liệu 56
Bảng 5.1 Bảng các model trackers của ultralytics 64
Bảng 6.1 Kết quả kiểm thử tự động 67
Bảng 6.2 Kết quả chức năng theo hai chế độ 69

Hình 3.1 Sơ đồ FAR/FRR theo threshold của hệ thống 37
Hình 3.2 Hình use case tổng quát 38
Hình 4.1. Kiến trúc triển khai của mã nguồn sản phẩm 49
Hình 4.2 Mô hình CSDL quan hệ 51
Hình 4.3 Chi tiết thuộc tính, khóa chính, khóa ngoại của cơ sở dữ liệu 52
Hình 4.4 Ranh giới lưu trữ giữa webcam và video tải lên 54
Hình 5.1 Đánh giá models phát hiện người 58
Hình 5.2 Đánh giá đo tốc độ xử lí của models phát hiện người 60
Hình 5.3 Đo FAR, FRR theo thresholds 61
Hình 5.4 Đánh giá đo tốc độ trích xuất embedding 62
Hình 5.5 Đánh giá các chỉ số cho các model trackers 63

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

Ký hiệu	Tên đầy đủ	Giải thích nghĩa
YOLO	You Only Look Once	Mô hình học sâu phát hiện vật thể/người thời gian thực trong một lần suy luận.
FPS	Frames Per Second	Số khung hình xử lý trên một giây, chỉ số đánh giá tốc độ/hiệu năng hệ thống.
FAR	False Acceptance Rate	Tỷ lệ chấp nhận nhầm (tỷ lệ người lạ bị nhận nhầm thành người hợp lệ)
FRR	False Rejection Rate	Tỷ lệ từ chối nhầm (tỷ lệ người đã đăng ký bị hệ thống từ chối/coi là người lạ).
EER	Equal Error Rate	Ngưỡng tỷ lệ lỗi cân bằng giữa FAR và FRR.
ROI	Region of Interest	Vùng quan tâm / Vùng đa giác cấu hình làm vùng cấm hoặc vùng giám sát
API	Application Programming Interface	Giao diện lập trình ứng dụng, dùng truyền nhận dữ liệu giữa Backend và Flutter.
JSON	JavaScript Object Notation	Định dạng chuẩn để trao đổi dữ liệu giữa các thành phần phần mềm.
CSDL / DB	Cơ sở dữ liệu / Database	Nơi lưu trữ thông tin hệ thống (dùng cơ sở dữ liệu SQLite).
CCTV	Closed-Circuit Television	Hệ thống camera giám sát truyền hình mạch kín.
FCM	Firebase Cloud Messaging	Dịch vụ điện toán đám mây của Google dùng để đẩy thông báo tới thiết bị di động
IoU	Intersection over Union	Chỉ số đánh giá tỷ lệ chồng lấp giữa hai vùng hộp giới hạn (bounding box).
TTL	Time To Live	Thời gian tồn tại của các dữ liệu/kết quả xử lý tạm thời.
RTSP	Real-Time Streaming Protocol	Giao thức truyền luồng video trực tiếp từ camera.

KẾ HOẠCH THỰC HIỆN CÔNG VIỆC NHÓM

TT	Nội dung	Người thực hiện	Thời gian thực hiện	Mức độ hoàn thành
1	Thu thập, tổ chức và tiền xử lý dữ liệu hình ảnh/video phục vụ đăng ký khuôn mặt và thử nghiệm mô hình	Bùi Mạnh Khôi	01/07-07/07	100%
2	Nghiên cứu, thử nghiệm pipeline AI trên Jupyter Notebook: phát hiện người/khuôn mặt, nhận dạng, tracking và phân tích hành vi cơ bản	Trần Tuấn Hải	01/07-11/07	100%
3	Thiết kế cơ sở dữ liệu và cấu trúc lưu trữ cục bộ: people, face_embeddings, cameras, surveillance_sessions, detection_logs, alerts, zones, ảnh, embedding, snapshot và upload tạm	Bùi Mạnh Khôi	01/07-28/07	100%
4	Xây dựng giao diện Flutter gồm Dashboard, Members, Register, Surveillance và Logs; kết nối giao diện với Backend API	Huỳnh Bá Anh Khoa	05/07-28/07	100%
5	Chuyển pipeline đăng ký khuôn mặt thành module Backend Python; xây dựng API đăng ký, danh sách, chi tiết và xóa thành viên	Huỳnh Bá Anh Khoa	09/07-28/07	100%
6	Tích hợp pipeline AI production gồm YOLOv8, ByteTrack, InsightFace, gallery embedding và recognition buffer vào Backend	Trần Tuấn Hải, Huỳnh Bá Anh Khoa	12/07-28/07	100%
7	Xây dựng quản lý camera, kiểm tra camera, snapshot preview, phiên live webcam, trạng thái giám sát, FPS, frame chú thích và kết quả nhận dạng mới nhất	Huỳnh Bá Anh Khoa, Trần Tuấn Hải	15/07-29/07	100%
8	Xây dựng cơ chế ghi nhận sự kiện live webcam: log người đã biết, xác nhận người lạ qua nhiều frame, chống cảnh báo trùng lặp và lưu snapshot cảnh	Trần Tuấn Hải, Bùi Mạnh Khôi	18/07-30/07	100%

	báo			
9	Xây dựng chức năng phân tích video tải lên: upload video, xử lý bằng job nền, hiển thị progress, FPS xử lý, frame chú thích, sự kiện tạm thời và tự xóa file sau xử lý.	Trần Tuấn Hải, Huỳnh Bá Anh Khoa	20/07-30/07	100%
10	Xây dựng chức năng vùng cấm: tạo, xem, cập nhật, xóa zone; chọn 4 điểm vùng trên giao diện Flutter; hiển thị vùng trên frame và sinh cảnh báo restricted_area khi người lạ ở trong vùng quá thời gian cấu hình.	Huỳnh Bá Anh Khoa, Trần Tuấn Hải	24/07-31/07	100%
11	Xây dựng màn hình Persistent Logs & Alerts: xem nhật ký, xem cảnh báo, đánh dấu đã đọc/chưa đọc, xóa log/alert và truy xuất snapshot cảnh báo.	Huỳnh Bá Anh Khoa	25/07-31/07	100%
12	Bổ sung readiness endpoint, script khởi động, Docker Compose, smoke test, runbook vận hành và kiểm tra ranh giới lưu trữ giữa live webcam và uploaded video.	Bùi Mạnh Khôi	26/07-31/07	100%
13	Kiểm thử tự động Backend và Flutter cho API, schema, camera, surveillance, video-analysis, logs, alerts, zones, recognition buffer và service contracts.	Huỳnh Bá Anh Khoa, Bùi Mạnh Khôi	28/07-02/08	100%
14	Kiểm thử thực tế ứng dụng với webcam, video tải lên và dữ liệu thực tế; kiểm tra giao diện, luồng thao tác, cảnh báo, snapshot, độ ổn định và khả năng sử dụng của hệ thống.	Huỳnh Bá Anh Khoa	01/08-07/08	100%
15	Xây dựng biểu đồ so sánh mô hình và đánh giá thực nghiệm: detector, recognition threshold, tốc độ trích xuất embedding, tracker, FPS, FAR/FRR và các chỉ số liên quan.	Trần Tuấn Hải, Bùi Mạnh Khôi	01/08-08/08	100%
16	Tổng hợp kết quả kiểm thử thực	Trần Tuấn Hải,	08/08-12/08	100%

	tế, kết quả so sánh mô hình, hình ảnh minh chứng, biểu đồ đánh giá và cập nhật báo cáo theo đúng trạng thái triển khai hiện tại.	Bùi Mạnh Khôi		
17	Hoàn thiện kế hoạch phát triển tiếp theo cho các chức năng chưa triển khai: loitering, rule engine cấu hình động, xác thực tài khoản, phân quyền, quản lý thiết bị và push notification.	Cả nhóm	Sau 12/08	Kế hoạch tiếp theo

MỞ ĐẦU

Trong những năm gần đây, sự phát triển nhanh chóng của trí tuệ nhân tạo, học sâu và thị giác máy tính đã mở ra nhiều hướng ứng dụng mới trong lĩnh vực giám sát an ninh. So với hệ thống camera truyền thống chủ yếu phục vụ việc ghi hình và tra cứu sau sự cố, hệ thống giám sát tích hợp trí tuệ nhân tạo có khả năng tự động phát hiện đối tượng, nhận dạng danh tính, theo dõi quá trình di chuyển và đưa ra cảnh báo khi xuất hiện các sự kiện bất thường. Xuất phát từ nhu cầu thực tế đó, nhóm lựa chọn thực hiện đề tài: "Xây dựng hệ thống nhận dạng khuôn mặt, giám sát an ninh và phân tích video thời gian thực".

Mục tiêu của đề tài là xây dựng một hệ thống giám sát có khả năng đăng ký và quản lý thông tin khuôn mặt, nhận dạng người đã đăng ký, phát hiện người lạ, theo dõi đối tượng trong luồng video và lưu lại các sự kiện phục vụ công tác quản lý. Hệ thống được thiết kế theo mô hình gồm ứng dụng Flutter ở phía người dùng, Backend FastAPI đảm nhiệm xử lý nghiệp vụ và tích hợp trí tuệ nhân tạo, cơ sở dữ liệu SQLite dùng để lưu trữ thông tin, cùng các mô hình YOLOv8, ByteTrack và InsightFace phục vụ phát hiện người, theo dõi đối tượng và nhận dạng khuôn mặt.

Trong quá trình thực hiện, nhóm đã chuyển pipeline thử nghiệm từ Jupyter Notebook thành các module sản phẩm có thể tích hợp vào Backend. Backend được tổ chức thành các module cấu hình, dữ liệu, AI, camera, dịch vụ và API. Cơ sở dữ liệu được mở rộng để quản lý thành viên, embedding khuôn mặt, camera, phiên giám sát, nhật ký nhận dạng, cảnh báo và vùng cấm. Pipeline YOLOv8, ByteTrack và InsightFace được đóng gói thành bộ máy phân tích khung hình dùng chung cho cả giám sát trực tiếp và phân tích video tải lên.

Hệ thống hiện hỗ trợ hai chế độ xử lý chính. Chế độ giám sát trực tiếp bằng webcam cho phép tạo phiên giám sát, hiển thị khung hình đã chú thích, nhận dạng người quen/người lạ, ghi nhận nhật ký, tạo cảnh báo người lạ đã được xác nhận và lưu snapshot làm minh chứng. Chế độ phân tích video tải lên sử dụng cùng pipeline AI nhưng chỉ lưu kết quả tạm thời trong bộ nhớ, bao gồm tiến độ xử lý, FPS, khung hình chú thích và danh sách sự kiện; tệp video được xóa sau khi xử lý và không ghi dữ liệu vào lịch sử giám sát.

Ứng dụng Flutter đã được kết nối với các API chính của Backend, bao gồm quản lý thành viên, quản lý camera, giám sát trực tiếp, phân tích video tải lên, nhật ký, cảnh báo và vùng cấm. Dashboard hiển thị trạng thái Backend, camera, số thành viên và cảnh báo chưa đọc; màn hình Register hỗ trợ đăng ký khuôn mặt; màn hình Members hỗ trợ quản lý thành viên; màn hình Surveillance tách rõ hai chế độ live webcam và uploaded video; màn hình Logs hiển thị nhật ký và cảnh báo bền vững của chế độ webcam.

Ngoài chức năng nhận dạng và cảnh báo người lạ, hệ thống đã triển khai chức năng vùng cấm ở mức cơ bản. Người dùng có thể tạo vùng bằng cách chọn bốn điểm trên giao diện Flutter, lưu vùng vào cơ sở dữ liệu và sử dụng vùng này để phát hiện trường hợp người lạ xuất hiện trong khu vực hạn chế quá thời gian cấu hình. Tuy nhiên, các chức năng nâng cao

như loitering, rule engine cấu hình động, xác thực tài khoản, phân quyền người dùng, quản lý thiết bị và push notification vẫn chưa thuộc phạm vi triển khai hoàn chỉnh của phiên bản hiện tại.

Báo cáo này trình bày tổng quan đề tài, cơ sở lý thuyết, phân tích yêu cầu, thiết kế hệ thống, kiến trúc triển khai, cơ sở dữ liệu, pipeline AI, kết quả tích hợp Flutter-FastAPI, kiểm thử tự động, kiểm thử thực tế với ứng dụng và dữ liệu thực tế, cũng như kết quả so sánh mô hình do nhóm thực hiện. Báo cáo đồng thời nêu rõ các giới hạn hiện tại của hệ thống nhằm phân biệt giữa chức năng đã được triển khai trong sản phẩm và các hướng phát triển tiếp theo.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1 Vấn đề cấp thiết

Camera giám sát hiện được sử dụng rộng rãi tại các cơ quan, doanh nghiệp, trường học, cửa hàng và khu vực công cộng. Tuy nhiên, phần lớn hệ thống camera truyền thống chủ yếu thực hiện chức năng ghi hình và lưu trữ dữ liệu. Việc theo dõi vẫn phụ thuộc nhiều vào con người, trong khi số lượng camera và lượng dữ liệu video phát sinh ngày càng lớn. Nhân viên giám sát khó có thể quan sát liên tục tất cả luồng video, dẫn đến nguy cơ bỏ sót người lạ, hành vi xâm nhập hoặc các sự kiện bất thường.

Khi sự cố xảy ra, dữ liệu từ camera truyền thống thường chỉ được sử dụng để truy xuất và kiểm tra lại. Cách tiếp cận này mang tính phản ứng sau sự cố, chưa hỗ trợ tốt việc phát hiện sớm và cảnh báo kịp thời. Vì vậy, nhu cầu xây dựng một hệ thống có khả năng tự động phân tích hình ảnh và video đang trở nên cần thiết.

Sự phát triển của trí tuệ nhân tạo, học sâu và thị giác máy tính đã tạo điều kiện để nâng cao khả năng của hệ thống giám sát. Các mô hình hiện nay có thể phát hiện người trong khung hình, xác định vị trí khuôn mặt, trích xuất đặc trưng nhận dạng, theo dõi đối tượng qua nhiều khung hình và hỗ trợ phát hiện một số tình huống bất thường dựa trên quy tắc. Việc kết hợp các kỹ thuật này giúp hệ thống camera chuyển từ chức năng ghi hình thụ động sang giám sát chủ động.

Đối với bài toán nhận dạng khuôn mặt, mỗi người được đăng ký bằng một hoặc nhiều hình ảnh khuôn mặt. Hệ thống kiểm tra ảnh đầu vào, phát hiện khuôn mặt, trích xuất đặc trưng dưới dạng vector embedding và lưu lại để phục vụ nhận dạng. Khi có người xuất hiện trong khung hình camera hoặc video, hệ thống so sánh embedding mới với dữ liệu đã đăng ký để xác định danh tính hoặc phân loại là người lạ.

Bên cạnh nhận dạng khuôn mặt, hệ thống giám sát thực tế còn phải xử lý luồng video liên tục, duy trì thông tin của một người qua nhiều khung hình và hạn chế việc tạo cảnh báo trùng lặp. Do đó, bài toán cần kết hợp nhiều thành phần như phát hiện người, nhận dạng khuôn mặt, theo dõi đối tượng, quản lý dữ liệu, lưu nhật ký, cảnh báo và giao tiếp giữa Backend với ứng dụng người dùng.

Từ những yêu cầu trên, nhóm lựa chọn thực hiện đề tài "Xây dựng hệ thống nhận dạng khuôn mặt, giám sát an ninh và phân tích video thời gian thực". Đề tài hướng đến việc xây dựng một sản phẩm thử nghiệm có khả năng đăng ký khuôn mặt, quản lý thành viên, nhận dạng người quen và người lạ từ webcam hoặc video tải lên, theo dõi đối tượng, lưu nhật ký sự kiện, tạo cảnh báo và hiển thị kết quả trên ứng dụng Flutter.

1.2 Nhiệm vụ và mục tiêu đề tài

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và xây dựng một hệ thống giám sát an ninh tích hợp trí tuệ nhân tạo. Hệ thống có khả năng tiếp nhận dữ liệu hình ảnh hoặc video, phát hiện và theo dõi người xuất hiện trong khung hình, nhận dạng khuôn mặt của thành viên đã đăng ký, phát hiện người lạ, tạo cảnh báo và cung cấp kết quả cho người quản lý thông qua giao diện ứng dụng.

Hệ thống được xây dựng theo kiến trúc gồm ứng dụng Flutter, Backend FastAPI, cơ sở dữ liệu SQLite và các module AI. Kiến trúc này giúp phân chia rõ ràng giữa giao diện người dùng, xử lý nghiệp vụ, lưu trữ dữ liệu và xử lý trí tuệ nhân tạo. Trong đó, Flutter đảm nhiệm hiển thị và thao tác người dùng; FastAPI cung cấp API và điều phối nghiệp vụ; SQLite lưu trữ dữ liệu cục bộ; các module AI thực hiện phát hiện người, theo dõi đối tượng và nhận dạng khuôn mặt.

1.2.2 Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, nhóm xác định các mục tiêu cụ thể như sau:

1. Nghiên cứu các kỹ thuật AI phục vụ giám sát:
   Tìm hiểu nguyên lý hoạt động và thử nghiệm các công nghệ gồm YOLOv8, ByteTrack và InsightFace. Trong đó, YOLOv8 được sử dụng để phát hiện người, ByteTrack dùng để theo dõi đối tượng qua nhiều khung hình, còn InsightFace buffalo_l được sử dụng để phát hiện khuôn mặt và trích xuất embedding phục vụ nhận dạng.
2. Xây dựng quy trình thu thập và tiền xử lý dữ liệu:
   Thu thập, phân loại và tổ chức dữ liệu hình ảnh/video phục vụ đăng ký khuôn mặt, kiểm thử ứng dụng và đánh giá mô hình. Dữ liệu đầu vào được kiểm tra định dạng, kích thước và chất lượng; ảnh khuôn mặt được xử lý trước khi trích xuất đặc trưng để lưu vào hệ thống.
3. Xây dựng chức năng đăng ký khuôn mặt:
   Cho phép người dùng nhập tên và lựa chọn nhiều ảnh khuôn mặt trên ứng dụng. Backend tiếp nhận ảnh, kiểm tra tính hợp lệ, phát hiện khuôn mặt, trích xuất embedding và lưu kết quả vào cơ sở dữ liệu cùng với đường dẫn ảnh và tệp embedding tương ứng.
4. Xây dựng cơ sở dữ liệu:
   Thiết kế cơ sở dữ liệu SQLite để quản lý các thành phần chính của hệ thống, bao gồm thành viên, ảnh khuôn mặt, embedding, camera, phiên giám sát, nhật ký nhận dạng, cảnh báo và vùng cấm. Cơ sở dữ liệu hiện sử dụng các bảng chính gồm people, face_embeddings, cameras, surveillance_sessions, detection_logs, alerts và zones.

Cách tổ chức này giúp hệ thống lưu trữ dữ liệu cần thiết cho chức năng đăng ký, giám sát trực tiếp, cảnh báo và truy xuất lịch sử.

5. Xây dựng Backend và API:
   Sử dụng Python và FastAPI để xây dựng các API giao tiếp với ứng dụng Flutter. Backend chịu trách nhiệm xử lý yêu cầu, gọi các module AI, truy xuất cơ sở dữ liệu, điều phối worker xử lý webcam/video và trả kết quả cho Frontend dưới dạng dữ liệu JSON hoặc frame ảnh đã chú thích.
6. Xây dựng giao diện ứng dụng:
   Phát triển ứng dụng Flutter gồm các màn hình Dashboard, Members, Register, Surveillance và Logs. Giao diện hỗ trợ quản lý thành viên, đăng ký khuôn mặt, quản lý camera, theo dõi trạng thái giám sát, phân tích video tải lên, cấu hình vùng cấm cơ bản và xem lại nhật ký hoặc cảnh báo.
7. Xử lý video và nhận dạng thời gian thực:
   Xây dựng pipeline đọc dữ liệu từ webcam hoặc video tải lên, phát hiện người bằng YOLOv8, theo dõi đối tượng bằng ByteTrack, nhận dạng khuôn mặt bằng InsightFace, phân loại kết quả thành known, unknown hoặc low_quality và chuyển kết quả xử lý đến ứng dụng.
8. Xây dựng cơ chế cảnh báo và lưu nhật ký:
   Khi phát hiện người lạ đã được xác nhận qua nhiều khung hình hoặc người lạ xuất hiện trong vùng cấm quá thời gian cấu hình, hệ thống tạo cảnh báo, lưu thời gian, kết quả nhận dạng và ảnh snapshot minh chứng. Người quản lý có thể xem lại thông tin trên ứng dụng thông qua màn hình Logs.
9. Kiểm thử và đánh giá hệ thống:
   Kiểm thử hệ thống bằng cả kiểm thử tự động và kiểm thử thực tế với ứng dụng, webcam, video tải lên và dữ liệu thực tế. Nhóm đánh giá các mô hình và pipeline thông qua notebook, biểu đồ so sánh và các chỉ số như Accuracy, FAR, FRR, Precision, Recall, F1-score, FPS và độ trễ xử lý.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm:

- Các phương pháp phát hiện người và khuôn mặt trong hình ảnh hoặc video.

- Phương pháp biểu diễn khuôn mặt bằng vector đặc trưng.

- Phương pháp so khớp embedding để nhận dạng người quen và người lạ.

- Thuật toán theo dõi đối tượng qua nhiều khung hình.

- Phương pháp phát hiện sự kiện an ninh dựa trên trạng thái nhận dạng và vùng giám sát.

- Quy trình thu thập, làm sạch và tổ chức dữ liệu khuôn mặt.

- Kiến trúc Backend xử lý AI và cung cấp API.

- Phương pháp thiết kế cơ sở dữ liệu cho hệ thống giám sát.

- Phương pháp kết nối ứng dụng Flutter với Backend FastAPI.

- Các chỉ số đánh giá độ chính xác và hiệu năng của hệ thống.

1.3.2 Phạm vi chức năng

Sản phẩm hoàn chỉnh được định hướng bao gồm những chức năng chính sau:

1. Đăng ký người dùng bằng một hoặc nhiều ảnh khuôn mặt.

2. Hiển thị và quản lý danh sách thành viên đã đăng ký.

3. Xóa thành viên cùng ảnh và embedding liên quan.

4. Quản lý camera phục vụ giám sát trực tiếp.

5. Tiếp nhận luồng video từ webcam hoặc tệp video tải lên.

6. Phát hiện người và khuôn mặt trong khung hình.

7. Theo dõi người qua nhiều khung hình liên tiếp.

8. Nhận dạng thành viên đã đăng ký.

9. Phân loại khuôn mặt không khớp thành người lạ.

10.Hiển thị frame đã chú thích và kết quả nhận dạng mới nhất.

11. Lưu nhật ký nhận dạng và ảnh minh chứng đối với chế độ webcam.

12. Tạo cảnh báo khi phát hiện người lạ đã được xác nhận.

13. Tạo cảnh báo vùng cấm khi người lạ xuất hiện trong vùng đã cấu hình quá thời gian quy định.

14. Hiển thị trạng thái hệ thống, kết quả giám sát, nhật ký và cảnh báo trên ứng dụng Flutter.

15. Hệ thống hiện tập trung vào sản phẩm thử nghiệm phục vụ mục đích nghiên cứu, kiểm thử và trình diễn. Các chức năng nâng cao như loitering, rule engine cấu hình động, xác thực tài khoản, phân quyền người dùng, quản lý thiết bị, push notification và xử lý đồng thời nhiều camera chưa thuộc phạm vi hoàn chỉnh của phiên bản hiện tại.

1.3.3 Phạm vi dữ liệu và môi trường

Dữ liệu được sử dụng gồm dữ liệu khuôn mặt công khai phục vụ thử nghiệm, hình ảnh do nhóm thu thập, dữ liệu thực tế dùng để kiểm thử ứng dụng và một số video ngắn dùng để kiểm tra pipeline. Dữ liệu được xử lý và lưu trữ cục bộ nhằm thuận tiện cho quá trình phát triển, kiểm thử và trình diễn.

Hệ thống ưu tiên hoạt động trong điều kiện ánh sáng tương đối ổn định, khuôn mặt có thể quan sát rõ và không bị che khuất nghiêm trọng. Hiệu quả nhận dạng có thể giảm khi khuôn mặt quá nhỏ, ảnh bị mờ, góc nghiêng lớn, người quay lưng hoặc điều kiện ánh sáng không phù hợp.

Phiên bản hiện tại sử dụng SQLite và lưu ảnh, embedding, snapshot trong các thư mục cục bộ. Camera chính được định hướng là webcam của máy tính hoặc nguồn camera cấu hình đơn giản. Chế độ video tải lên hỗ trợ các định dạng phổ biến như MP4, AVI, MOV, MKV và WebM. Những nội dung như triển khai nhiều camera IP đồng thời, xử lý phân tán, lưu trữ đám mây, xác thực người dùng và thông báo đẩy chưa phải là mục tiêu ưu tiên của phiên bản hiện tại.

1.3.4 Phạm vi của báo cáo hiện tại

Tại thời điểm hoàn thiện báo cáo, nhóm đã đạt được các kết quả chính sau:

- Ổn định cấu trúc Backend FastAPI, cấu hình đường dẫn, cơ chế khởi tạo tài nguyên và tổ chức mã nguồn theo các module routes, services, AI, camera và database.

- Xây dựng bộ máy AI dùng chung gồm phát hiện người bằng YOLOv8, theo dõi đối tượng bằng ByteTrack, nhận dạng khuôn mặt bằng InsightFace và bộ đệm nhận dạng theo từng track.

- Hoàn thiện chức năng đăng ký và quản lý thành viên, bao gồm đăng ký khuôn mặt từ nhiều ảnh, lưu ảnh khuôn mặt, lưu embedding, hiển thị danh sách thành viên, xem chi tiết và xóa thành viên.

- Hoàn thiện quản lý camera và vòng đời giám sát trực tiếp từ webcam, bao gồm thêm camera, kiểm tra camera, lấy snapshot preview, bắt đầu/dừng phiên giám sát, hiển thị FPS, số frame đã xử lý, frame chú thích và kết quả nhận dạng mới nhất.

- Tích hợp giao diện Flutter với Backend cho các màn hình Dashboard, Members, Register, Surveillance và Logs. Giao diện hiện hỗ trợ hai chế độ xử lý là live webcam và uploaded video.

- Mở rộng SQLite lên schema phiên bản 2 với các bảng people, face_embeddings, cameras, surveillance_sessions, detection_logs, alerts và zones.

- Xây dựng cơ chế lưu trữ rõ ràng giữa hai chế độ: live webcam lưu phiên giám sát, nhật ký, cảnh báo và snapshot; uploaded video chỉ lưu kết quả tạm thời trong bộ nhớ, không ghi lịch sử vào SQLite và xóa tệp sau khi xử lý.

- Triển khai chức năng vùng cấm ở mức cơ bản, cho phép người dùng chọn bốn điểm trên giao diện Flutter, lưu zone vào cơ sở dữ liệu và tạo cảnh báo restricted_area khi người lạ xuất hiện trong vùng quá thời gian cấu hình.

- Bổ sung readiness endpoint, script khởi động, Docker Compose, smoke test và tài liệu vận hành để hỗ trợ triển khai, kiểm tra và chạy thử hệ thống.

Kết quả kiểm thử hiện đạt 108 kiểm thử Backend và 13 kiểm thử Flutter; Flutter static analysis không phát hiện lỗi. Ngoài ra, nhóm đã thực hiện kiểm thử thực tế ứng dụng với webcam, video tải lên và dữ liệu thực tế. Tuy nhiên, các chức năng nâng cao như loitering, rule engine cấu hình động, xác thực tài khoản, phân quyền, quản lý thiết bị và push notification vẫn thuộc hướng phát triển tiếp theo.

1.4 Phương pháp nghiên cứu và quy trình thực hiện

1.4.1 Phương pháp nghiên cứu

Nhóm kết hợp các phương pháp nghiên cứu sau:

1. Phương pháp nghiên cứu tài liệu:

Nhóm tìm hiểu tài liệu về thị giác máy tính, phát hiện đối tượng, nhận dạng khuôn mặt, embedding, cosine similarity và theo dõi đa đối tượng. Tài liệu kỹ thuật của Python, FastAPI, Flutter, SQLite, OpenCV, YOLOv8, ByteTrack và InsightFace được sử dụng để lựa chọn công nghệ và xây dựng hệ thống.

2. Phương pháp thực nghiệm:

Các mô hình và thuật toán được thử nghiệm độc lập trong Jupyter Notebook trước khi tích hợp vào sản phẩm. Nhóm sử dụng dữ liệu hình ảnh và video để đánh giá khả năng phát hiện, nhận dạng và theo dõi, từ đó lựa chọn pipeline phù hợp cho hệ thống.

3. Phương pháp phát triển phần mềm:

Hệ thống được chia thành các module gồm Frontend, Backend, AI, camera, dịch vụ và cơ sở dữ liệu. Các thành phần được phát triển riêng biệt, sau đó kết nối thông qua API để hình thành luồng xử lý hoàn chỉnh từ đăng ký khuôn mặt, giám sát webcam, phân tích video tải lên đến hiển thị nhật ký và cảnh báo.

4. Phương pháp kiểm thử:

Nhóm kiểm tra từng chức năng độc lập trước khi kiểm thử tích hợp. Đối với chức năng đăng ký, hệ thống được kiểm tra với ảnh hợp lệ, ảnh không đọc được, ảnh không có khuôn mặt, ảnh có nhiều khuôn mặt và ảnh có kích thước quá nhỏ. Đối với chức năng giám sát và video, nhóm kiểm tra trạng thái worker, frame chú thích, kết quả nhận dạng, log, alert, snapshot, vùng cấm và ranh giới lưu trữ giữa webcam với video tải lên.

5. Phương pháp đánh giá định lượng:

Nhóm đánh giá các mô hình và pipeline AI thông qua kết quả thử nghiệm trong Jupyter Notebook, biểu đồ so sánh mô hình và kiểm thử thực tế với dữ liệu hình ảnh/video. Các chỉ số được sử dụng hoặc định hướng sử dụng gồm Accuracy, Precision, Recall, F1-score, FAR, FRR, FPS và độ trễ xử lý. Kết quả này giúp nhóm lựa chọn mô hình phù hợp và xác định các giới hạn cần cải thiện trong giai đoạn tiếp theo.

1.4.2 Quy trình thực hiện

Quy trình thực hiện đề tài được chia thành năm giai đoạn:

Giai đoạn 1 - Khảo sát và xác định yêu cầu:

> Nhóm xác định mục tiêu, chức năng, công nghệ và kiến trúc tổng thể của hệ thống. Các chức năng được phân chia thành đăng ký khuôn mặt, quản lý thành viên, quản lý camera, giám sát trực tiếp, phân tích video tải lên, nhận dạng, cảnh báo, vùng cấm và nhật ký.

Giai đoạn 2 - Chuẩn bị dữ liệu và thử nghiệm AI:

> Dữ liệu được thu thập, phân loại và tiền xử lý. Các mô hình YOLOv8, ByteTrack và InsightFace được nghiên cứu, thử nghiệm và đánh giá trong Jupyter Notebook. Nhóm cũng xây dựng các biểu đồ so sánh mô hình để phục vụ lựa chọn pipeline và trình bày kết quả thực nghiệm.

Giai đoạn 3 - Xây dựng cơ sở dữ liệu, Backend và giao diện:

> Nhóm thiết kế cơ sở dữ liệu SQLite, xây dựng Backend FastAPI và phát triển giao diện Flutter. Các chức năng nền tảng như đăng ký khuôn mặt, quản lý thành viên, quản lý camera và kết nối Frontend-Backend được triển khai trước để kiểm tra khả năng hoạt động của toàn bộ hệ thống.

Giai đoạn 4 - Tích hợp hệ thống:

> Pipeline AI từ Notebook được chuyển thành các module Python có thể tái sử dụng trong Backend. Backend sử dụng YOLOv8, ByteTrack và InsightFace để xử lý frame từ webcam hoặc video tải lên. Frontend gửi yêu cầu qua API, Backend xử lý dữ liệu, lưu kết quả cần thiết và phản hồi để giao diện cập nhật trạng thái, frame chú thích, kết quả nhận dạng, nhật ký và cảnh báo.

Giai đoạn 5 - Kiểm thử, đánh giá và hoàn thiện:

> Hệ thống được kiểm thử bằng kiểm thử tự động và kiểm thử thực tế với webcam, video tải lên và dữ liệu thực tế. Nhóm đo lường độ chính xác, tốc độ xử lý, độ trễ và độ ổn định của ứng dụng; sửa lỗi giao diện, xử lý lỗi Backend và điều chỉnh ngưỡng nhận dạng khi cần thiết. Cuối cùng, nhóm hoàn thiện tài liệu, báo cáo, sản phẩm trình diễn và định hướng phát triển tiếp theo cho các chức năng nâng cao như

loitering, rule engine cấu hình động, xác thực tài khoản, phân quyền và push notification.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG

2.1 Tổng quan về trí tuệ nhân tạo, học máy và học sâu

Trí tuệ nhân tạo (Artificial Intelligence - AI) là lĩnh vực nghiên cứu và phát triển các hệ thống có khả năng thực hiện những nhiệm vụ thường yêu cầu trí thông minh của con người, chẳng hạn như nhận biết hình ảnh, xử lý ngôn ngữ, dự đoán và ra quyết định. Trong đề tài này, AI được ứng dụng để tự động phát hiện người, nhận dạng khuôn mặt, theo dõi đối tượng và phân tích sự kiện từ dữ liệu hình ảnh hoặc video.

Học máy (Machine Learning - ML) là một nhánh của AI, trong đó hệ thống học các quy luật từ dữ liệu thay vì chỉ hoạt động dựa trên các điều kiện được lập trình thủ công. Một mô hình học máy được huấn luyện bằng dữ liệu mẫu, sau đó sử dụng các đặc trưng đã học để đưa ra dự đoán đối với dữ liệu mới.

Học sâu (Deep Learning - DL) là một nhánh của học máy sử dụng mạng nơ-ron có nhiều tầng. Các mô hình học sâu có khả năng tự động học đặc trưng từ dữ liệu thô. Trong lĩnh vực thị giác máy tính, các tầng đầu của mạng thường học những đặc trưng cơ bản như cạnh, góc và màu sắc, trong khi các tầng sâu hơn có thể học được cấu trúc phức tạp như khuôn mặt hoặc hình dáng con người.

Mạng nơ-ron tích chập (Convolutional Neural Network - CNN) là kiến trúc được sử dụng phổ biến trong xử lý ảnh. Phép tích chập cho phép mô hình khai thác thông tin không gian và nhận biết đặc trưng cục bộ trong ảnh. CNN là nền tảng của nhiều mô hình phát hiện đối tượng, phát hiện khuôn mặt và nhận dạng khuôn mặt hiện đại.

Trong hệ thống của đề tài, các kỹ thuật học sâu được sử dụng tại ba nhóm chức năng chính:

1. Phát hiện người và khuôn mặt trong hình ảnh hoặc video.

2. Trích xuất vector đặc trưng để nhận dạng khuôn mặt.

3. Theo dõi một người qua nhiều khung hình video liên tiếp.

Các kết quả từ mô hình AI được kết hợp với cơ sở dữ liệu, trạng thái theo dõi đối tượng và các quy tắc nghiệp vụ để tạo thành hệ thống giám sát có khả năng nhận dạng, ghi nhật ký và tạo cảnh báo.

2.2 Xử lý ảnh, phát hiện người và khuôn mặt

2.2.1 Tiền xử lý và kiểm tra chất lượng ảnh

Tiền xử lý là bước chuẩn bị dữ liệu trước khi ảnh được đưa vào mô hình AI. Chất lượng dữ liệu đầu vào ảnh hưởng trực tiếp đến khả năng phát hiện và nhận dạng khuôn mặt. Ảnh bị mờ, quá nhỏ, sai định dạng, không thể đọc được hoặc chứa nhiều khuôn mặt có thể tạo ra embedding không ổn định và làm giảm độ chính xác của hệ thống.

Quy trình tiền xử lý ảnh trong đề tài gồm các bước chính:

1. Kiểm tra tên và định dạng tệp.
2. Kiểm tra dữ liệu ảnh có đọc được hay không.
3. Kiểm tra kích thước tối thiểu của ảnh.
4. Phát hiện số lượng khuôn mặt bằng mô hình InsightFace.
5. Loại bỏ ảnh không có khuôn mặt, có nhiều khuôn mặt hoặc khuôn mặt quá nhỏ.
6. Cắt vùng ảnh chứa khuôn mặt.
7. Thay đổi kích thước ảnh khuôn mặt về kích thước thống nhất.
8. Trích xuất vector đặc trưng và lưu embedding.

Hệ thống hiện hỗ trợ các định dạng ảnh thông dụng như JPG, JPEG, PNG và WebP. Ảnh có kích thước quá nhỏ hoặc không thể giải mã sẽ bị từ chối trước khi đưa vào mô hình nhận dạng. Đối với ảnh đăng ký, yêu cầu mỗi ảnh chỉ có một khuôn mặt. Quy tắc này giúp bảo đảm embedding được lưu thuộc đúng người cần đăng ký. Nếu không phát hiện được khuôn mặt, phát hiện nhiều khuôn mặt hoặc khuôn mặt không đạt kích thước tối thiểu, Backend trả về lý do từ chối cụ thể để người dùng lựa chọn ảnh khác.

Trong quá trình thử nghiệm dữ liệu trên Jupyter Notebook, nhóm có thể sử dụng phương sai của toán tử Laplacian để hỗ trợ xác định ảnh mờ. Với ảnh xám I, mức độ sắc nét có thể được ước lượng bằng:

$$B = \mathrm{Var}\left(\nabla^2 I\right)$$

Trong đó:

- $\nabla^2 I$ là kết quả áp dụng toán tử Laplacian lên ảnh;
- Var là phép tính phương sai;
- B là điểm độ sắc nét của ảnh.

Nếu B nhỏ hơn một ngưỡng được thiết lập, ảnh có thể được xem là ảnh mờ. Ngưỡng này cần được điều chỉnh theo độ phân giải và điều kiện thu thập ảnh thực tế.

2.2.2 Phát hiện khuôn mặt bằng InsightFace và SCRFD

Phát hiện khuôn mặt là quá trình xác định vị trí của một hoặc nhiều khuôn mặt trong ảnh. Kết quả phát hiện thường được biểu diễn bằng hộp giới hạn:

$$Box = (x, y, w, h)$$

Trong đó x, y là tọa độ góc trên bên trái, còn w, h lần lượt là chiều rộng và chiều cao của khuôn mặt.

Trong phiên bản Backend hiện tại, hệ thống sử dụng InsightFace buffalo_l để phát hiện khuôn mặt và trích xuất embedding. InsightFace tích hợp bộ phát hiện khuôn mặt dựa trên SCRFD, có khả năng phát hiện khuôn mặt trong nhiều điều kiện khác nhau về kích thước, góc nhìn và ánh sáng. Khi đăng ký thành viên, Backend gọi mô hình InsightFace để kiểm tra số lượng khuôn mặt trong từng ảnh. Ảnh chỉ được chấp nhận khi mô hình phát hiện đúng một khuôn mặt hợp lệ.

So với các phương pháp cổ điển như Haar Cascade, SCRFD có khả năng thích nghi tốt hơn trong các tình huống khuôn mặt có kích thước khác nhau, góc nhìn thay đổi hoặc điều kiện ánh sáng không đồng đều. Vì vậy, hệ thống hiện ưu tiên sử dụng InsightFace/SCRFD thay vì chỉ dựa vào bộ phân loại truyền thống.

Kết quả phát hiện khuôn mặt không chỉ phục vụ việc kiểm tra ảnh đầu vào mà còn được dùng để cắt vùng khuôn mặt, trích xuất embedding và lưu dữ liệu nhận dạng cho từng thành viên.

2.2.3 Căn chỉnh và cắt khuôn mặt

Khuôn mặt trong ảnh có thể xuất hiện ở nhiều vị trí, kích thước và góc nghiêng khác nhau. Nếu đưa trực tiếp vùng ảnh chưa chuẩn hóa vào mô hình, embedding thu được có thể không ổn định. Vì vậy, khuôn mặt cần được cắt và chuẩn hóa trước khi lưu trữ hoặc sử dụng cho nhận dạng.

Quy trình cơ bản gồm: Ảnh gốc -> Phát hiện khuôn mặt -> Xác định bounding box -> Mở rộng vùng cắt -> Chuẩn hóa kích thước -> Lưu ảnh khuôn mặt và embedding

Trong Backend hiện tại, bounding box được lấy từ kết quả phát hiện của InsightFace. Trước khi cắt ảnh, hệ thống mở rộng bounding box thêm một khoảng đệm để tránh cắt mất các vùng quan trọng của khuôn mặt. Tọa độ sau khi mở rộng được giới hạn trong kích thước ảnh gốc để tránh vượt khỏi biên ảnh.

Ảnh khuôn mặt sau khi cắt được chuyển sang hệ màu RGB và thay đổi về kích thước thống nhất. Backend hiện lưu ảnh khuôn mặt đã cắt ở kích thước 160 x 160 pixel. Trong các thử nghiệm notebook, ảnh khuôn mặt cũng có thể được căn chỉnh theo năm điểm mốc và chuẩn hóa về kích thước 112 x 112 pixel để phù hợp với một số quy trình đánh giá ArcFace.

Năm điểm mốc thường được sử dụng gồm:

1. Tâm mắt trái.
2. Tâm mắt phải.
3. Đỉnh mũi.
4. Khóe miệng trái.
5. Khóe miệng phải.

Từ các điểm mốc này, hệ thống có thể sử dụng phép biến đổi affine để xoay, dịch chuyển và chuẩn hóa khuôn mặt về một tư thế gần với ảnh mẫu. Tuy nhiên, trong sản phẩm hiện tại, phần quan trọng nhất là bảo đảm khuôn mặt được phát hiện đúng, cắt đúng vùng và tạo embedding ổn định cho quá trình so khớp.

2.2.4 Phát hiện người bằng YOLOv8

YOLO (You Only Look Once) là một họ mô hình phát hiện đối tượng theo thời gian thực. Thay vì xử lý ảnh qua nhiều bước độc lập, YOLO thực hiện dự đoán vị trí và lớp của đối tượng trong một lần suy luận. Điều này giúp YOLO phù hợp với các bài toán cần tốc độ xử lý cao như giám sát camera hoặc phân tích video.

YOLOv8 có thể phát hiện nhiều loại đối tượng. Trong đề tài, hệ thống chỉ quan tâm đến lớp person. Mỗi kết quả phát hiện bao gồm:

1. Tọa độ bounding box.

2. Nhãn đối tượng.

3. Độ tin cậy.

4. Vị trí đối tượng trong khung hình.

Luồng xử lý sử dụng YOLOv8 trong hệ thống được xây dựng như sau:

Khung hình -> YOLOv8 phát hiện người -> ByteTrack gán track_id -> Cắt vùng người -> InsightFace phát hiện khuôn mặt trong vùng người -> Trích xuất embedding -> So khớp với gallery

Cách tiếp cận này giúp hệ thống phát hiện và theo dõi một người ngay cả khi khuôn mặt của họ tạm thời không quan sát được. Khi khuôn mặt xuất hiện rõ trong vùng ảnh của người, module nhận dạng mới được thực hiện. Trong phiên bản hiện tại, YOLOv8 và ByteTrack không chỉ được thử nghiệm trong Jupyter Notebook mà đã được tích hợp vào Backend thông qua bộ máy phân tích khung hình dùng chung.

2.3 Trích xuất và so khớp đặc trưng khuôn mặt

2.3.1 Biểu diễn khuôn mặt bằng embedding

Trong hệ thống nhận dạng khuôn mặt, ảnh không được so sánh trực tiếp theo từng điểm ảnh. Thay vào đó, mô hình học sâu chuyển mỗi khuôn mặt thành một vector đặc trưng gọi là embedding:

$$e = f(I)$$

Trong đó, I là ảnh khuôn mặt, f là mô hình trích xuất đặc trưng và e là embedding thu được. Các ảnh của cùng một người thường tạo ra các embedding gần nhau trong không gian vector, trong khi embedding của những người khác nhau có khoảng cách lớn hơn.

Đề tài sử dụng InsightFace buffalo_l để phát hiện khuôn mặt và trích xuất embedding. Trong chức năng đăng ký, mỗi ảnh hợp lệ tạo ra một embedding riêng. Embedding được lưu dưới dạng tệp NumPy (.npy), còn đường dẫn đến ảnh khuôn mặt và embedding được lưu trong cơ sở dữ liệu SQLite. Khi hệ thống bắt đầu giám sát hoặc phân tích video, các embedding đã đăng ký được nạp vào gallery trong bộ nhớ để phục vụ so khớp nhanh.

2.3.2 Nguyên lý ArcFace

ArcFace là phương pháp được sử dụng để tăng khả năng phân biệt giữa các khuôn mặt. Phương pháp này bổ sung một biên góc vào hàm mất mát trong quá trình huấn luyện, giúp embedding của cùng một người nằm gần nhau và embedding của những người khác nhau cách xa nhau hơn.

Trong đề tài, nhóm không huấn luyện ArcFace từ đầu mà sử dụng mô hình pretrained thuộc bộ InsightFace buffalo_l. Cách tiếp cận này giúp giảm thời gian huấn luyện, tận dụng mô

hình đã được huấn luyện trên tập dữ liệu lớn và phù hợp với phạm vi của sản phẩm thử nghiệm. Nhiệm vụ chính của hệ thống là khai thác embedding từ mô hình pretrained, lưu trữ embedding của người đã đăng ký và so khớp embedding mới khi có người xuất hiện trong camera hoặc video.

2.3.3 So khớp bằng cosine similarity

Sau khi trích xuất embedding của khuôn mặt cần nhận dạng, hệ thống so sánh vector này với các embedding đã đăng ký bằng cosine similarity.

Giá trị similarity càng lớn thì hai khuôn mặt càng có khả năng thuộc cùng một người. Hệ thống chọn người có điểm similarity cao nhất và so sánh kết quả với một ngưỡng T:

Nếu similarity >= T: khuôn mặt được xác định là người đã đăng ký.

Nếu similarity < T: khuôn mặt được phân loại là Unknown.

Nếu không phát hiện được khuôn mặt rõ trong vùng người: kết quả được đánh dấu là low_quality hoặc kế thừa danh tính đã biết trước đó của cùng track nếu có.

Ngưỡng T cần được lựa chọn dựa trên quá trình thực nghiệm. Ngưỡng quá thấp có thể làm tăng khả năng nhận nhầm người lạ, trong khi ngưỡng quá cao có thể khiến hệ thống từ chối người đã đăng ký. Trong hệ thống hiện tại, kết quả so khớp còn được đưa qua recognition buffer theo từng track_id để giảm dao động nhận dạng giữa các khung hình liên tiếp.

2.4 Theo dõi đối tượng và phân tích hành vi

2.4.1 Theo dõi đa đối tượng bằng ByteTrack

Phát hiện đối tượng chỉ xác định người xuất hiện trong từng khung hình riêng lẻ. Để theo dõi cùng một người qua nhiều khung hình, hệ thống cần sử dụng thuật toán theo dõi đa đối tượng.

ByteTrack nhận kết quả phát hiện người từ YOLOv8 và gán một mã track_id cho mỗi đối tượng. Mã này được duy trì khi người đó tiếp tục xuất hiện trong các khung hình sau. ByteTrack sử dụng cả các detection có độ tin cậy cao và thấp để hạn chế mất dấu khi đối tượng bị che khuất hoặc chất lượng hình ảnh giảm.

Khi một khuôn mặt trong track được nhận dạng thành công, danh tính có thể được gắn với track_id thông qua bộ đệm nhận dạng. Nhờ đó, nếu khuôn mặt tạm thời không quan sát được ở một vài khung hình tiếp theo, hệ thống vẫn có thể duy trì trạng thái nhận dạng đã biết của track thay vì lập tức kết luận là người lạ hoặc ảnh kém chất lượng.

Trong phiên bản hiện tại, YOLOv8 và ByteTrack đã được tích hợp vào Backend. Cả chế độ live webcam và uploaded video đều sử dụng cùng một bộ máy phân tích khung hình, trong đó mỗi phiên xử lý có trạng thái tracker và recognition buffer riêng để tránh lẫn dữ liệu giữa các lần chạy.

2.4.2 Phân tích hành vi dựa trên quy tắc

Hệ thống sử dụng phương pháp dựa trên quy tắc để phát hiện một số sự kiện an ninh. Quy tắc được xây dựng dựa trên trạng thái nhận dạng, vị trí đối tượng, vùng giám sát và thời gian xuất hiện của đối tượng.

Trong phiên bản hiện tại, hệ thống đã triển khai các quy tắc chính sau:

1. Phát hiện người lạ đã được xác nhận qua nhiều khung hình.

2. Chống tạo cảnh báo trùng lặp cho cùng một track trong thời gian ngắn.

3. Phát hiện người lạ xuất hiện trong vùng cấm quá thời gian cấu hình.

4. Không tạo cảnh báo người lạ từ các khung hình low_quality hoặc không phát hiện được khuôn mặt rõ.

Vùng giám sát được biểu diễn bằng tập hợp các điểm tạo thành vùng cấm. Trên giao diện Flutter, người dùng có thể chọn bốn điểm để tạo zone cho camera. Backend lưu zone vào cơ sở dữ liệu và sử dụng vùng này trong quá trình giám sát trực tiếp. Khi một người lạ đã được xác nhận có bounding box giao với vùng cấm và tồn tại trong vùng quá thời gian dwell cấu hình, hệ thống tạo cảnh báo restricted_area và lưu snapshot minh chứng.

Các chức năng phân tích hành vi nâng cao như loitering tổng quát, rule engine cấu hình động, phát hiện đám đông hoặc phát hiện camera mất kết nối dưới dạng cảnh báo riêng vẫn thuộc hướng phát triển tiếp theo.

2.5 Công nghệ phát triển hệ thống

2.5.1 Backend FastAPI

FastAPI là framework Python được sử dụng để xây dựng Backend và cung cấp API cho ứng dụng Flutter. Backend chịu trách nhiệm tiếp nhận yêu cầu, xử lý ảnh, gọi các module AI, truy xuất cơ sở dữ liệu, điều phối worker xử lý webcam/video và trả kết quả dưới dạng JSON hoặc ảnh JPEG đã chú thích.

Các nhóm API hiện đã được xây dựng gồm:

GET /health: kiểm tra trạng thái Backend.

GET /health/readiness: kiểm tra trạng thái cơ sở dữ liệu, thư mục lưu trữ, model AI và worker.

POST /members/register: đăng ký thành viên bằng ảnh khuôn mặt.

GET /members: lấy danh sách thành viên.

GET /members/{id}: lấy thông tin chi tiết của một thành viên.

DELETE /members/{id}: xóa thành viên và dữ liệu liên quan.

POST /cameras: thêm camera.

GET /cameras: lấy danh sách camera.

GET /cameras/{id}: lấy thông tin một camera.

PATCH /cameras/{id}: cập nhật hoặc kích hoạt lại camera.

DELETE /cameras/{id}: vô hiệu hóa camera.

POST /cameras/{id}/test: kiểm tra camera.

GET /cameras/{id}/snapshot: lấy ảnh preview từ camera.

POST /surveillance/start: bắt đầu giám sát trực tiếp.

POST /surveillance/stop: dừng giám sát trực tiếp.

GET /surveillance/status: lấy trạng thái giám sát.

GET /surveillance/latest: lấy kết quả nhận dạng mới nhất.

GET /surveillance/frame: lấy frame chú thích mới nhất.

GET /logs và GET /logs/latest: xem nhật ký nhận dạng.

GET /alerts, GET /alerts/latest và GET /alerts/{id}: xem cảnh báo.

PATCH /alerts/{id}/read: đánh dấu cảnh báo đã đọc hoặc chưa đọc.

GET /alerts/{id}/snapshot: xem snapshot cảnh báo.

POST /video-analysis: tải video lên để phân tích.

GET /video-analysis/{job_id}/status: xem trạng thái xử lý video.

GET /video-analysis/{job_id}/results: xem kết quả phân tích video tạm thời.

GET /video-analysis/{job_id}/frame: xem frame chú thích mới nhất của video.

DELETE /video-analysis/{job_id}: hủy hoặc xóa kết quả job video.

POST /zones, GET /zones, PATCH /zones/{id}, DELETE /zones/{id}: quản lý vùng cấm.

Backend được tổ chức theo hướng tách trách nhiệm rõ ràng. Routes xử lý API, schemas định nghĩa dữ liệu vào/ra, services xử lý nghiệp vụ, module AI xử lý phân tích hình ảnh/video, camera module mở và kiểm tra nguồn camera, còn database module quản lý SQLite schema và kết nối dữ liệu.

2.5.2 Giao diện Flutter

Flutter được sử dụng để phát triển giao diện đa nền tảng của hệ thống. Ứng dụng gồm các màn hình chính: Dashboard, Members, Register, Surveillance và Logs.

Màn hình Dashboard hiển thị trạng thái Backend, số lượng thành viên, camera, cảnh báo chưa đọc và thông tin mới nhất của hệ thống. Màn hình Register cho phép người dùng nhập tên và chọn nhiều ảnh để đăng ký khuôn mặt. Màn hình Members hiển thị danh sách thành viên đã đăng ký, chi tiết ảnh khuôn mặt và hỗ trợ xóa thành viên.

Màn hình Surveillance là nơi người dùng thao tác với hai chế độ xử lý. Chế độ live webcam cho phép chọn camera, thêm laptop webcam, bắt đầu/dừng giám sát, xem frame chú thích, xem kết quả nhận dạng hiện tại và cấu hình vùng cấm bằng cách chọn bốn điểm trên frame. Chế độ uploaded video cho phép chọn tệp video, theo dõi trạng thái xử lý, xem tiến độ, FPS, frame chú thích và danh sách sự kiện tạm thời.

Màn hình Logs hiển thị dữ liệu bền vững của chế độ webcam, bao gồm detection logs và alerts. Người dùng có thể xem cảnh báo, đánh dấu đã đọc/chưa đọc, xóa log/alert và truy xuất snapshot minh chứng. Kết quả của chế độ uploaded video không được đưa vào lịch sử này để bảo đảm ranh giới lưu trữ rõ ràng giữa hai chế độ xử lý.

2.5.3 Cơ sở dữ liệu SQLite

SQLite được lựa chọn vì có cấu trúc đơn giản, không yêu cầu máy chủ riêng và phù hợp với sản phẩm thử nghiệm. Phiên bản hiện tại sử dụng cơ sở dữ liệu cục bộ để lưu các dữ liệu cần thiết cho đăng ký khuôn mặt, quản lý camera, giám sát trực tiếp, nhật ký, cảnh báo và vùng cấm.

Các bảng chính của hệ thống gồm:

1. people: lưu thông tin thành viên đã đăng ký.

2. face_embeddings: lưu đường dẫn ảnh khuôn mặt và embedding tương ứng của từng thành viên.

3. cameras: lưu thông tin camera, nguồn camera, vị trí và trạng thái hoạt động.

4. surveillance_sessions: lưu phiên giám sát trực tiếp bằng webcam.

5. detection_logs: lưu các sự kiện nhận dạng có ý nghĩa trong chế độ webcam.

6. alerts: lưu cảnh báo, loại cảnh báo, thông điệp, snapshot và trạng thái đã đọc/chưa đọc.

7. zones: lưu vùng cấm theo từng camera dưới dạng tọa độ điểm.

Ảnh khuôn mặt, embedding, snapshot cảnh báo và tệp upload tạm thời được lưu trong các thư mục cục bộ. Cơ sở dữ liệu chỉ lưu thông tin và đường dẫn đến tệp, giúp việc lưu trữ, truy xuất và dọn dẹp dữ liệu thuận tiện hơn.

Hệ thống có ranh giới lưu trữ rõ ràng giữa live webcam và uploaded video. Live webcam có thể tạo surveillance_sessions, detection_logs, alerts và snapshot. Uploaded video chỉ giữ tiến độ, frame và sự kiện trong bộ nhớ tạm thời; tệp video được xóa sau khi hoàn tất, lỗi hoặc hủy job; kết quả video không được ghi vào lịch sử giám sát.

2.6 Các chỉ số đánh giá hệ thống

2.6.1 Chỉ số đánh giá nhận dạng

Accuracy là tỷ lệ dự đoán đúng trên tổng số mẫu:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Precision là tỷ lệ dự đoán đúng trong tổng số kết quả được dự đoán thuộc lớp cần quan tâm:

$$\text{Precision} = TP / (TP + FP)$$

Recall là tỷ lệ mẫu đúng được phát hiện trên tổng số mẫu thực tế:

$$\text{Recall} = TP / (TP + FN)$$

F1-score là trung bình điều hòa giữa Precision và Recall:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

False Acceptance Rate (FAR) là tỷ lệ người lạ bị nhận nhầm thành người đã đăng ký. False Rejection Rate (FRR) là tỷ lệ người đã đăng ký bị từ chối hoặc bị phân loại thành người lạ. Hai chỉ số này được sử dụng để lựa chọn ngưỡng nhận dạng phù hợp.

Trong đề tài, các chỉ số này được sử dụng trong quá trình đánh giá mô hình và lựa chọn threshold nhận dạng. Việc lựa chọn threshold cần cân bằng giữa an toàn và khả năng sử dụng: threshold quá thấp có thể tăng nguy cơ nhận nhầm người lạ, còn threshold quá cao có thể làm tăng số trường hợp người quen bị nhận dạng sai thành unknown.

2.6.2 Chỉ số đánh giá phát hiện và theo dõi

Intersection over Union (IoU) đo mức độ chồng lấp giữa bounding box dự đoán và bounding box thực tế:

IoU = Diện tích phần giao / Diện tích phần hợp

Đối với phát hiện người hoặc khuôn mặt, IoU được dùng để đánh giá độ chính xác vị trí của bounding box. Ngoài IoU, các chỉ số như Precision, Recall và F1-score cũng có thể được dùng để đánh giá khả năng phát hiện đúng đối tượng.

Đối với theo dõi đối tượng, hệ thống có thể sử dụng các chỉ số MOTA, IDF1 và ID Switch. Trong đó, ID Switch là số lần hệ thống gán sai mã định danh của một đối tượng khi người đó di chuyển, bị che khuất hoặc giao cắt với người khác. Các chỉ số này giúp đánh giá khả năng duy trì track_id ổn định qua nhiều khung hình.

Trong quá trình thực hiện, nhóm xây dựng các biểu đồ so sánh mô hình và kết quả thử nghiệm để đánh giá detector, mô hình nhận dạng và tracker. Các kết quả này giúp lựa chọn pipeline phù hợp cho sản phẩm thử nghiệm.

2.6.3 Chỉ số hiệu năng

FPS là số khung hình hệ thống xử lý được trong một giây:

FPS = Số khung hình / Tổng thời gian xử lý

Độ trễ là khoảng thời gian từ lúc camera thu nhận khung hình đến khi kết quả được hiển thị trên ứng dụng. Hiệu năng phụ thuộc vào cấu hình phần cứng, độ phân giải video, số lượng người trong khung hình, tốc độ đọc camera và số mô hình được chạy trên mỗi khung hình.

Để giảm tải, hệ thống sử dụng cách tiếp cận theo track: YOLOv8 phát hiện người, ByteTrack duy trì track_id, InsightFace nhận dạng khuôn mặt khi có vùng mặt đủ rõ, sau đó recognition buffer giúp ổn định kết quả qua nhiều khung hình. Cách tổ chức này giúp

hạn chế dao động nhận dạng và hỗ trợ xử lý video theo thời gian thực tốt hơn trong phạm vi sản phẩm thử nghiệm.

2.7. Kết luận chương

Chương 2 đã trình bày các cơ sở lý thuyết và công nghệ chính được sử dụng trong đề tài, bao gồm trí tuệ nhân tạo, học máy, học sâu, xử lý ảnh, phát hiện khuôn mặt bằng InsightFace/SCRFD, phát hiện người bằng YOLOv8, embedding khuôn mặt, ArcFace, cosine similarity, theo dõi đối tượng bằng ByteTrack, phân tích sự kiện dựa trên quy tắc, FastAPI, Flutter và SQLite.

Các thành phần AI đảm nhiệm việc phân tích hình ảnh và video; Backend điều phối quá trình xử lý, quản lý dữ liệu và tạo API; ứng dụng Flutter cung cấp giao diện tương tác cho người dùng; SQLite và hệ thống thư mục cục bộ lưu trữ dữ liệu cần thiết cho đăng ký, giám sát, nhật ký và cảnh báo. Những nội dung này là nền tảng để phân tích yêu cầu, thiết kế hệ thống và đánh giá kết quả triển khai trong các chương tiếp theo.

CHƯƠNG 3: PHÂN TÍCH YÊU CẦU HỆ THỐNG

3.1 Yêu cầu chức năng

3.1.1 Tiếp nhận và xử lý luồng video thời gian thực

Hệ thống cần tiếp nhận dữ liệu hình ảnh/video từ hai nguồn chính: camera webcam trực tiếp và tệp video do người dùng tải lên. Đối với chế độ live webcam, hệ thống mở kết nối tới camera đã cấu hình, đọc từng khung hình, đưa vào pipeline AI để phát hiện người, theo dõi đối tượng, nhận dạng khuôn mặt và trả về kết quả mới nhất cho giao diện Flutter.

Đối với chế độ uploaded video, người dùng chọn tệp video từ thiết bị, Backend tiếp nhận tệp, tạo job xử lý nền và cập nhật tiến độ xử lý cho giao diện. Kết quả của video tải lên bao gồm trạng thái job, số frame đã xử lý, FPS xử lý, frame đã chú thích và danh sách sự kiện tạm thời.

Hệ thống cần bảo đảm tại một thời điểm chỉ có một pipeline xử lý nặng hoạt động, nhằm tránh việc live webcam và uploaded video cùng sử dụng tài nguyên AI gây giảm hiệu năng. Live webcam được lưu lịch sử giám sát, còn uploaded video chỉ giữ kết quả tạm thời trong bộ nhớ và xóa tệp sau khi xử lý.

3.1.2 Đăng ký thông tin và lưu trữ thư viện khuôn mặt (Enrollment)

Người dùng có thể đăng ký thành viên bằng cách nhập tên và tải lên một hoặc nhiều ảnh khuôn mặt. Backend kiểm tra định dạng ảnh, kích thước ảnh, khả năng đọc ảnh và số lượng khuôn mặt trong ảnh. Ảnh hợp lệ là ảnh có đúng một khuôn mặt rõ ràng và đạt kích thước tối thiểu.

Sau khi ảnh hợp lệ, hệ thống sử dụng InsightFace để phát hiện khuôn mặt, trích xuất vector embedding và lưu dữ liệu vào hệ thống. Ảnh khuôn mặt sau khi cắt được lưu trong thư mục cục bộ, embedding được lưu dưới dạng tệp NumPy, còn đường dẫn đến ảnh và embedding được lưu trong cơ sở dữ liệu SQLite.

Thư viện khuôn mặt được sử dụng khi hệ thống chạy live webcam hoặc phân tích video. Các embedding đã đăng ký được nạp vào gallery trong bộ nhớ để phục vụ so khớp nhanh với khuôn mặt mới xuất hiện trong khung hình.

3.1.3 Nhận dạng danh tính và phát hiện người lạ

Hệ thống cần phát hiện người trong khung hình, cắt vùng người, phát hiện khuôn mặt trong vùng người và trích xuất embedding khuôn mặt. Embedding mới được so khớp với gallery embedding đã đăng ký bằng cosine similarity.

Kết quả nhận dạng được phân loại thành ba trạng thái chính:

1. Known: khuôn mặt khớp với một thành viên đã đăng ký.

2. Unknown: khuôn mặt được phát hiện nhưng không khớp với thành viên nào theo ngưỡng nhận dạng.

3. Low_quality: không phát hiện được khuôn mặt đủ rõ hoặc frame không đủ chất lượng để kết luận.

Để giảm dao động nhận dạng giữa các frame liên tiếp, hệ thống sử dụng recognition buffer theo từng track_id. Khi một track đã được nhận dạng ổn định, hệ thống có thể duy trì danh tính đó trong một số tình huống khuôn mặt tạm thời không quan sát được.

3.1.4 Thiết lập vùng cấm và cảnh báo hành vi bất thường

Hệ thống cho phép người dùng thiết lập vùng cấm ở mức cơ bản trên giao diện Surveillance. Người dùng chọn bốn điểm trên khung hình live webcam để tạo thành một vùng giám sát. Vùng này được lưu vào bảng zones và có thể được bật, tắt hoặc xóa.

Trong phiên bản hiện tại, hệ thống đã hỗ trợ cảnh báo người lạ và cảnh báo vùng cấm. Cảnh báo người lạ chỉ được tạo khi một track giữ trạng thái unknown qua số frame xác nhận cấu hình, giúp hạn chế cảnh báo sai. Cảnh báo restricted_area được tạo khi người lạ đã xác nhận xuất hiện trong vùng cấm quá thời gian dwell cấu hình.

Các hành vi nâng cao như loitering tổng quát, rule engine cấu hình động, phát hiện đám đông hoặc các loại hành vi bất thường phức tạp chưa thuộc phạm vi hoàn chỉnh của phiên bản hiện tại và được đưa vào hướng phát triển tiếp theo.

3.1.5 Hiển thị và xử lý cảnh báo trên ứng dụng

Sau khi hệ thống phát hiện sự kiện cần cảnh báo, Backend lưu thông tin cảnh báo, loại cảnh báo, thời gian, camera, trạng thái nhận dạng, độ tin cậy nếu có và snapshot minh chứng. Người dùng có thể xem lại cảnh báo trên màn hình Logs của ứng dụng Flutter.

Ứng dụng hiện hỗ trợ xem danh sách cảnh báo, xem snapshot cảnh báo, đánh dấu cảnh báo đã đọc/chưa đọc và xóa cảnh báo. Push notification qua Firebase Cloud Messaging hoặc cơ chế thông báo nền tới thiết bị di động chưa được triển khai trong phiên bản hiện tại.

3.2 Yêu cầu phi chức năng

3.2.1 Hiệu năng và tốc độ xử lý

Hệ thống cần xử lý video theo hướng gần thời gian thực, trong đó giao diện có thể liên tục nhận frame chú thích, trạng thái giám sát, số frame đã xử lý và FPS. Hiệu năng phụ thuộc

vào cấu hình phần cứng, độ phân giải video, số lượng người trong khung hình và tốc độ của các mô hình AI.

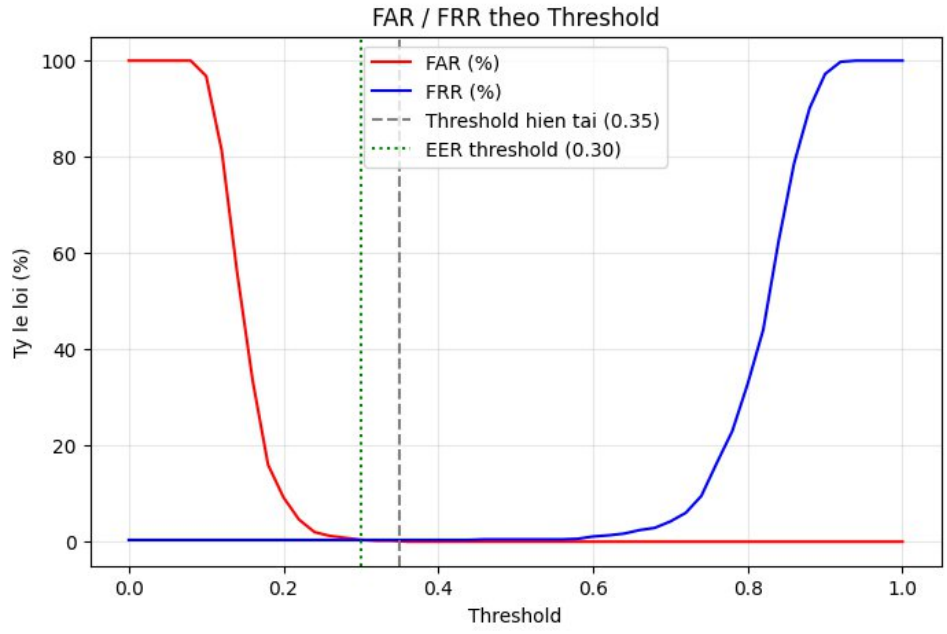
Đối với live webcam, hệ thống cần duy trì vòng lặp đọc frame, phân tích frame và cập nhật kết quả mới nhất cho Flutter polling. Đối với uploaded video, hệ thống cần cập nhật tiến độ xử lý, FPS xử lý và trạng thái job để người dùng biết quá trình phân tích đang diễn ra.

Trong phiên bản thử nghiệm, mục tiêu quan trọng là bảo đảm pipeline hoạt động ổn định, tránh chạy đồng thời nhiều tác vụ AI nặng và cung cấp đủ thông tin chẩn đoán như trạng thái worker, lỗi camera, lỗi video và readiness.

3.2.2 Độ tin cậy và tỷ lệ cảnh báo sai (False Alarm Rate)

Hệ thống cần giảm cảnh báo sai bằng nhiều cơ chế. Đối với người lạ, Backend không tạo cảnh báo ngay tại frame đầu tiên mà yêu cầu track giữ trạng thái unknown qua nhiều frame liên tiếp. Đối với frame chất lượng thấp hoặc không phát hiện được khuôn mặt rõ, hệ thống không tự động tạo cảnh báo người lạ.

Ngưỡng nhận dạng ảnh hưởng trực tiếp đến FAR và FRR. Nếu threshold quá thấp, người lạ có thể bị nhận nhầm thành người đã đăng ký, làm tăng False Acceptance Rate. Nếu threshold quá cao, người đã đăng ký có thể bị từ chối hoặc phân loại thành unknown, làm tăng False Rejection Rate.



Hình 3.1 Sơ đồ FAR/FRR theo threshold của hệ thống

Càng tăng threshold thì tỷ lệ nhận nhầm người lạ thường giảm, nhưng tỷ lệ từ chối nhầm người hợp lệ có thể tăng. Vì vậy, threshold cần được lựa chọn dựa trên kết quả thực nghiệm, dữ liệu thực tế và mục tiêu an toàn của hệ thống.

3.3 Kịch bản sử dụng hệ thống (Use Case Diagram & Đặc tả chi tiết)

3.3.1 Danh sách Actor

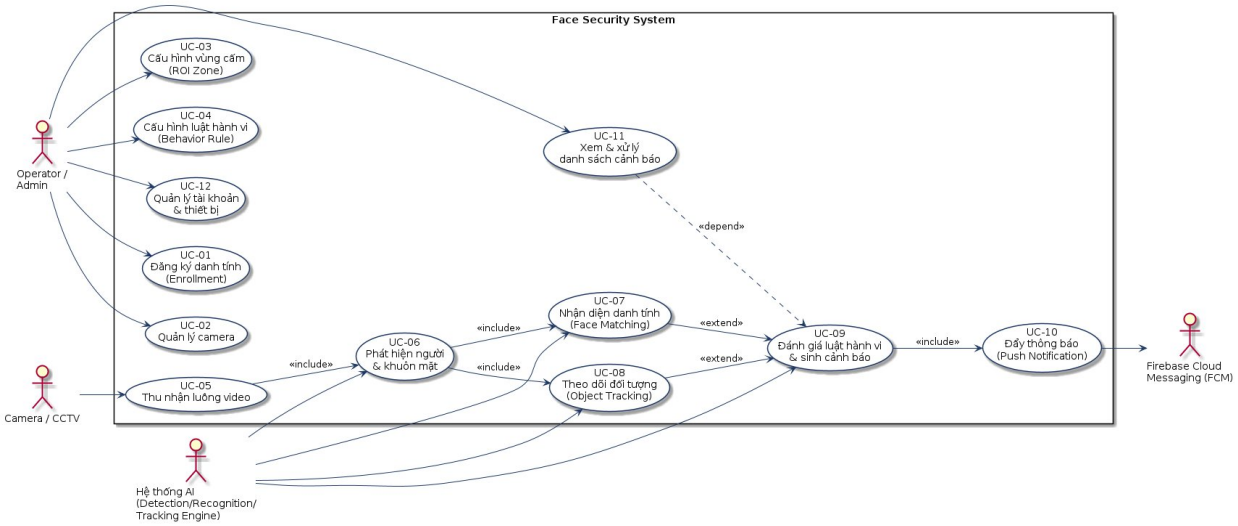
- Operator/Admin: người vận hành hệ thống qua ứng dụng Flutter, thực hiện đăng ký danh tính, quản lý thành viên, quản lý camera, cấu hình vùng cấm, bắt đầu/dừng giám sát, phân tích video tải lên và xem nhật ký/cảnh báo.

- Camera/Webcam: thiết bị hoặc nguồn đầu vào cung cấp khung hình video cho hệ thống giám sát trực tiếp.

- Hệ thống AI (Detection/Recognition/Tracking Engine): thành phần Backend tự động chạy YOLOv8, ByteTrack và InsightFace để phát hiện người, theo dõi đối tượng, nhận dạng khuôn mặt và sinh kết quả phân tích.

- SQLite Database: thành phần lưu trữ dữ liệu thành viên, embedding, camera, phiên giám sát, nhật ký, cảnh báo và vùng cấm.

3.3.2 Sơ đồ Use Case tổng quan



Hình 3.2 Hình use case tổng quát

3.3.3 Đặc tả chi tiết từng Use Case

* UC-01 — Đăng ký danh tính (Enrollment)

Mã Use Case	UC-01
Tên Use Case	Đăng ký danh tính (Enrollment)
Actor	Operator/Admin
Mô tả tóm tắt	Operator đăng ký một người mới vào hệ thống bằng cách nhập tên và tải lên một hoặc nhiều ảnh khuôn mặt để hệ thống trích xuất embedding phục vụ nhận dạng.
Tiền điều kiện	Có ít nhất một ảnh khuôn mặt rõ ràng, đúng định dạng và đủ kích thước.
Hậu điều kiện	Một bản ghi mới được tạo trong bảng people; ảnh khuôn mặt và embedding được lưu vào hệ thống; thông tin đường dẫn được lưu trong bảng face_embeddings.
Luồng sự kiện chính	1. Operator mở màn hình Register. 2. Nhập tên người cần đăng ký. 3. Chọn một hoặc nhiều ảnh khuôn mặt. 4. Backend kiểm tra định dạng, kích thước và khả năng đọc ảnh. 5. InsightFace phát hiện khuôn mặt và trích xuất embedding. 6. Backend lưu ảnh khuôn mặt, embedding và thông tin thành viên. 7. Ứng dụng hiển thị kết quả ảnh được chấp nhận hoặc bị từ chối.
Luồng thay thế	Nếu một số ảnh không hợp lệ, hệ thống vẫn đăng ký các ảnh hợp lệ và trả về danh sách ảnh bị từ chối kèm lý do.
Ngoại lệ	Ảnh rỗng, sai định dạng, không đọc được, quá nhỏ, không có khuôn mặt hoặc có nhiều khuôn mặt sẽ bị từ chối.
Liên quan	Bảng people, face_embeddings; màn hình Register, Members.

Bảng 3.1 Bảng use case đăng ký danh tính

* UC-02 — Quản lý camera

Mã Use Case	UC-02
Tên Use Case	Quản lý camera
Actor	Operator/Admin
Mô tả tóm tắt	Operator thêm, xem, cập nhật, kiểm tra hoặc vô hiệu hóa camera trong hệ thống.
Tiền điều kiện	Backend đang hoạt động; nguồn camera có thể là webcam hoặc nguồn camera được cấu hình.
Hậu điều kiện	Thông tin camera được lưu hoặc cập nhật trong bảng cameras; camera active có thể được chọn để giám sát.
Luồng sự kiện chính	1. Operator mở màn hình Surveillance. 2. Thêm laptop webcam hoặc chọn camera đã có. 3. Backend lưu thông tin camera gồm tên, source, location và trạng thái active. 4. Operator có thể test camera hoặc lấy snapshot preview.
Luồng thay thế	Camera có thể được cập nhật hoặc soft-delete bằng cách chuyển is_active = 0 để không còn xuất hiện trong danh sách mặc định.
Ngoại lệ	Camera không tồn tại, không active hoặc đang được live surveillance sử dụng thì một số thao tác preview/test sẽ bị từ chối.
Liên quan	Bảng cameras; API /cameras; màn hình Surveillance.

Bảng 3.2 Bảng use case quản lý camera

* UC-03 — Cấu hình vùng cấm (ROI Zone)

Mã Use Case	UC-03
Tên Use Case	Cấu hình vùng cấm (ROI Zone)
Actor	Operator/Admin
Mô tả tóm tắt	Operator tạo vùng cấm cho một camera bằng cách chọn bốn điểm trên khung hình live webcam.
Tiền điều kiện	Camera đã được tạo và live webcam đang có frame để người dùng chọn điểm.

Mã Use Case	UC-03
Hậu điều kiện	Vùng cấm được lưu vào bảng zones dưới dạng tọa độ điểm và có thể được dùng khi giám sát trực tiếp.
Luồng sự kiện chính	1. Operator mở chế độ Live webcam. 2. Bắt đầu giám sát camera. 3. Chọn bốn điểm trên frame để tạo vùng. 4. Nhập tên vùng. 5. Backend lưu zone vào cơ sở dữ liệu. 6. Vùng được hiển thị trên frame chú thích.
Luồng thay thế	Operator có thể undo điểm, xóa vùng nháp, bật/tắt vùng hoặc xóa vùng đã lưu.
Ngoại lệ	Tên vùng rỗng, trùng tên trên cùng camera hoặc vùng không đủ bốn điểm sẽ bị từ chối.
Liên quan	Bảng zones; API /zones; màn hình Surveillance.

Bảng 3.3 bảng use case cấu hình vùng cấm

* UC-04 — Cấu hình luật hành vi nâng cao

Mã Use Case	UC-04
Tên Use Case	Cấu hình luật hành vi nâng cao
Actor	Operator/Admin
Mô tả tóm tắt	Operator định nghĩa các luật cảnh báo nâng cao như loitering, intrusion hoặc rule engine cấu hình động.
Tiền điều kiện	Hệ thống đã có camera, vùng giám sát và dữ liệu tracking.
Hậu điều kiện	Trong định hướng tương lai, luật hành vi sẽ được lưu và áp dụng tự động khi giám sát.
Luồng sự kiện chính	1. Operator chọn loại luật. 2. Chọn vùng áp dụng hoặc toàn khung hình. 3. Nhập ngưỡng thời gian hoặc ngưỡng tin cậy. 4. Hệ thống lưu luật.

Mã Use Case	UC-04
	5. Worker giám sát đọc luật để sinh cảnh báo.
Luồng thay thế	Operator có thể bật/tắt luật mà không xóa cấu hình.
Ngoại lệ	Ngưỡng không hợp lệ hoặc vùng không tồn tại sẽ bị từ chối.
Liên quan	Đây là chức năng kế hoạch tiếp theo. Phiên bản hiện tại mới có logic cố định cho unknown_person và restricted_area, chưa có rule engine cấu hình động.

Bảng 3.4 Bảng use case cấu hình luật hành vi

* UC-05 — Thu nhận luồng video

Mã Use Case	UC-05
Tên Use Case	Thu nhận luồng video
Actor	Operator/Admin, Camera/Webcam
Mô tả tóm tắt	Backend mở nguồn camera đang active và bắt đầu phiên giám sát trực tiếp để xử lý frame theo thời gian thực.
Tiền điều kiện	Camera tồn tại, đang active và không có uploaded-video job đang chạy.
Hậu điều kiện	Một bản ghi surveillance_sessions được tạo; worker live surveillance bắt đầu đọc frame và cập nhật kết quả mới nhất.
Luồng sự kiện chính	1. Operator chọn camera. 2. Nhấn Start. 3. Backend tạo surveillance session. 4. Worker mở camera và đọc frame liên tục. 5. Frame được đưa vào pipeline AI. 6. Giao diện nhận status, FPS, frame chú thích và kết quả nhận dạng.
Luồng thay thế	Operator nhấn Stop để dừng giám sát; Backend release camera và cập nhật session thành stopped.
Ngoại lệ	Camera không tồn tại, không active, không mở được hoặc mất frame liên tục thì session được đánh dấu failed và trả lỗi cho giao diện.

Mã Use Case	UC-05
Liên quan	Bảng cameras, surveillance_sessions; API /surveillance.

Bảng 3.5 Bảng use case thu nhận luồng video

* UC-06 — Phát hiện người và khuôn mặt

Mã Use Case	UC-06
Tên Use Case	Phát hiện người và khuôn mặt
Actor	Hệ thống AI
Mô tả tóm tắt	Với mỗi frame, hệ thống dùng YOLOv8 để phát hiện người, sau đó dùng InsightFace để phát hiện khuôn mặt trong vùng người.
Tiền điều kiện	Có frame hợp lệ từ webcam hoặc video tải lên.
Hậu điều kiện	Hệ thống tạo kết quả phân tích frame gồm track_id, bounding box, confidence, trạng thái nhận dạng và thông tin khuôn mặt nếu có.
Luồng sự kiện chính	1. Worker nhận frame. 2. YOLOv8 phát hiện người. 3. ByteTrack gán track_id. 4. Backend cắt vùng người. 5. InsightFace phát hiện khuôn mặt trong vùng người. 6. Kết quả được chuyển sang bước nhận dạng.
Luồng thay thế	Nếu không phát hiện được khuôn mặt trong vùng người, track được đánh dấu low_quality hoặc kế thừa danh tính đã biết trước đó nếu có.
Ngoại lệ	Frame rỗng hoặc sai định dạng sẽ bị từ chối bởi engine phân tích.
Liên quan	Module FrameAnalysisEngine, YoloByteTracker, InsightFaceTrackRecognizer.

Bảng 3.6 Bảng use case phát hiện người và khuôn mặt

* UC-07 — Nhận diện danh tính (Face Matching)

Mã Use Case	UC-07
Tên Use Case	Nhận diện danh tính (Face Matching)
Actor	Hệ thống AI
Mô tả tóm tắt	Hệ thống so khớp embedding khuôn mặt mới với gallery embedding đã đăng ký để xác định known hoặc unknown.
Tiền điều kiện	Có khuôn mặt được phát hiện và gallery embedding đã được nạp từ cơ sở dữ liệu.
Hậu điều kiện	Track được gán trạng thái known, unknown hoặc low_quality. Nếu known, kết quả có member_id và tên thành viên.
Luồng sự kiện chính	1. InsightFace trích xuất embedding. 2. Hệ thống tính cosine similarity với gallery. 3. Chọn ứng viên có similarity cao nhất. 4. So sánh với threshold cấu hình. 5. Đưa kết quả qua recognition buffer để ổn định nhận dạng.
Luồng thay thế	Nếu gallery rỗng hoặc similarity dưới threshold, khuôn mặt được phân loại unknown.
Ngoại lệ	Nếu không trích xuất được embedding hợp lệ, kết quả được xem là low_quality.
Liên quan	Bảng people, face_embeddings; module InMemoryFaceGallery, recognition buffer.

Bảng 3.7 Bảng use case nhận diện danh tính

* UC-08 — Theo dõi đối tượng (Object Tracking)

Mã Use Case	UC-08
Tên Use Case	Theo dõi đối tượng (Object Tracking)
Actor	Hệ thống AI
Mô tả tóm tắt	Hệ thống sử dụng ByteTrack để duy trì track_id cho mỗi người qua nhiều frame liên tiếp.

Mã Use Case	UC-08
Tiền điều kiện	YOLOv8 phát hiện được ít nhất một người trong frame.
Hậu điều kiện	Mỗi người được gán track_id; trạng thái nhận dạng được quản lý theo từng track.
Luồng sự kiện chính	1. YOLOv8 trả về bounding box người. 2. ByteTrack liên kết detection mới với track đang tồn tại. 3. Nếu là người mới, hệ thống tạo track_id mới. 4. Recognition buffer lưu trạng thái nhận dạng theo track_id. 5. Kết quả được trả về cho live webcam hoặc uploaded video.
Luồng thay thế	Khi người tạm thời quay lưng hoặc khuôn mặt không rõ, track vẫn có thể được duy trì để tránh mất trạng thái ngay lập tức.
Ngoại lệ	ID switch có thể xảy ra khi người bị che khuất hoặc giao cắt; đây là chỉ số cần theo dõi trong đánh giá tracker.
Liên quan	YOLOv8, ByteTrack, recognition buffer.

Bảng 3.8 Bảng use case theo dõi đối tượng

* UC-09 — Đánh giá luật hành vi và sinh cảnh báo

Mã Use Case	UC-09
Tên Use Case	Đánh giá sự kiện và sinh cảnh báo
Actor	Hệ thống AI, Backend
Mô tả tóm tắt	Backend đánh giá kết quả nhận dạng/tracking để quyết định có lưu log hoặc tạo cảnh báo hay không.
Tiền điều kiện	Có kết quả phân tích frame từ live webcam.
Hậu điều kiện	Hệ thống lưu detection_logs, alerts và snapshot nếu sự kiện thỏa điều kiện cảnh báo.
Luồng sự kiện chính	1. LiveEventRecorder nhận kết quả frame. 2. Nếu track known mới xuất hiện hoặc đổi danh tính, hệ thống lưu log. 3. Nếu track unknown đủ số frame xác nhận, hệ thống tạo unknown_person alert.

Mã Use Case	UC-09
	4. Nếu unknown track nằm trong vùng cấm quá dwell time, hệ thống tạo restricted_area alert. 5. Snapshot frame chú thích được lưu làm minh chứng.
Luồng thay thế	Nếu cùng một track tiếp tục vi phạm trong thời gian ngắn, hệ thống áp dụng cooldown/chống trùng lặp để không tạo quá nhiều alert.
Ngoại lệ	Low_quality frame không làm tăng unknown streak và không tạo cảnh báo người lạ.
Liên quan	Bảng detection_logs, alerts, zones; service LiveEventRecorder.

Bảng 3.9 Bảng use case đánh giá luật hành vi & sinh ra cảnh báo

* UC-10 — Đẩy thông báo (Push Notification)

Mã Use Case	UC-10
Tên Use Case	Đẩy thông báo (Push Notification)
Actor	Backend, Firebase Cloud Messaging
Mô tả tóm tắt	Backend gửi thông báo tới thiết bị người dùng khi có cảnh báo mới.
Tiền điều kiện	Có alert mới và thiết bị người dùng đã đăng ký token thông báo.
Hậu điều kiện	Người dùng nhận thông báo trên thiết bị.
Luồng sự kiện chính	1. Backend tạo alert. 2. Backend lấy danh sách device token. 3. Backend gửi payload qua FCM. 4. Thiết bị nhận và hiển thị thông báo.
Luồng thay thế	Nếu không có thiết bị active, alert vẫn được lưu để người dùng xem trong ứng dụng.
Ngoại lệ	Token không hợp lệ hoặc FCM lỗi thì Backend ghi nhận lỗi và không gửi lại token không hợp lệ.
Liên quan	Chức năng này chưa được triển khai trong phiên bản hiện tại; hiện tại cảnh

Mã Use Case	UC-10
	báo được xem trực tiếp trong ứng dụng Flutter.

Bảng 3.10 Bảng use case đẩy thông báo

* UC-11 — Xem và xử lý danh sách cảnh báo

Mã Use Case	UC-11
Tên Use Case	Xem và xử lý danh sách cảnh báo
Actor	Operator/Admin
Mô tả tóm tắt	Operator xem danh sách nhật ký và cảnh báo đã lưu từ chế độ live webcam.
Tiền điều kiện	Hệ thống đã tạo detection log hoặc alert trong quá trình giám sát webcam.
Hậu điều kiện	Người dùng có thể xem, đánh dấu đã đọc/chưa đọc hoặc xóa log/alert.
Luồng sự kiện chính	1. Operator mở màn hình Logs. 2. Ứng dụng gọi API lấy detection logs và alerts. 3. Người dùng xem thời gian, camera, trạng thái, thông điệp và snapshot nếu có. 4. Người dùng đánh dấu alert đã đọc/chưa đọc.
Luồng thay thế	Người dùng có thể xóa một hoặc nhiều log/alert, hoặc xóa toàn bộ danh sách.
Ngoại lệ	Nếu snapshot không tồn tại hoặc đường dẫn không hợp lệ, Backend trả lỗi phù hợp và không cho truy xuất ngoài thư mục snapshot.
Liên quan	Bảng detection_logs, alerts; API /logs, /alerts; màn hình Logs.

Bảng 3.11 Bảng use case xem và xử lý danh sách cảnh báo

* UC-12 — Quản lý tài khoản và thiết bị

Mã Use Case	UC-12
Tên Use Case	Quản lý tài khoản và thiết bị
Actor	Operator/Admin
Mô tả tóm tắt	Operator đăng nhập, phân quyền và đăng ký thiết bị để phục vụ thông báo.

Mã Use Case	UC-12
Tiền điều kiện	Hệ thống có bảng users, phân quyền và cơ chế xác thực.
Hậu điều kiện	Người dùng đăng nhập thành công và thiết bị được liên kết với tài khoản.
Luồng sự kiện chính	1. Người dùng nhập tài khoản/mật khẩu. 2. Backend xác thực thông tin. 3. Ứng dụng nhận quyền truy cập. 4. Thiết bị gửi token thông báo lên Backend.
Luồng thay thế	Người dùng có thể đăng xuất hoặc cập nhật token thiết bị.
Ngoại lệ	Sai thông tin đăng nhập hoặc tài khoản bị khóa thì hệ thống từ chối truy cập.
Liên quan	Chức năng này chưa được triển khai trong phiên bản hiện tại; hệ thống hiện tập trung vào sản phẩm thử nghiệm cục bộ.

Bảng 3.12 Bảng use case quản lý tài khoản và thiết bị

3.4 Đối chiếu yêu cầu với phạm vi triển khai hiện tại

Các Use Case ở Chương 3 mô tả cả chức năng đã triển khai và các chức năng định hướng cho hệ thống hoàn chỉnh. Trong phiên bản hiện tại, nhóm đã hiện thực các luồng cốt lõi gồm UC-01, UC-02, UC-03, UC-05, UC-06, UC-07, UC-08, UC-09 và UC-11.

Cụ thể, hệ thống đã hỗ trợ đăng ký danh tính, quản lý thành viên, quản lý camera, cấu hình vùng cấm cơ bản, giám sát live webcam, phân tích uploaded video, phát hiện người, nhận dạng khuôn mặt, theo dõi đối tượng, lưu nhật ký, tạo cảnh báo unknown_person, tạo cảnh báo restricted_area và xem/xử lý cảnh báo trên ứng dụng Flutter.

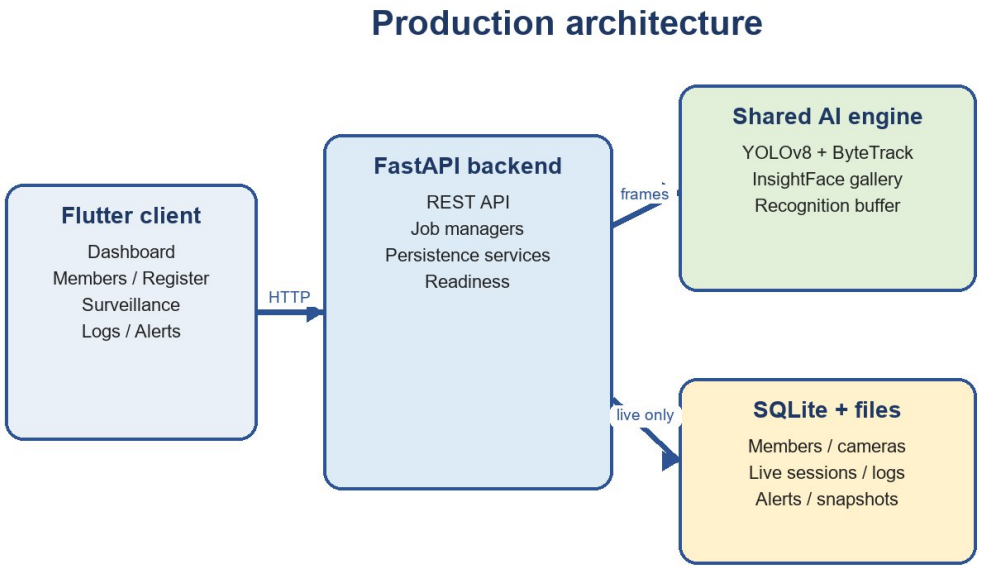
Các chức năng UC-04, UC-10 và UC-12 vẫn thuộc phạm vi phát triển tiếp theo. Phiên bản hiện tại chưa có rule engine cấu hình động, chưa có loitering tổng quát, chưa có xác thực tài khoản, phân quyền, quản lý thiết bị hoặc Firebase Cloud Messaging. Việc đối chiếu này giúp phân biệt rõ chức năng đã được kiểm chứng trong mã nguồn với các yêu cầu mở rộng của hệ thống trong tương lai.

CHƯƠNG 4: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

4.1 Kiến trúc mã nguồn sản phẩm

Mã nguồn sản phẩm được tổ chức theo hướng tách biệt giữa phần Backend, Frontend, AI, dữ liệu runtime và tài liệu. Thư mục backend/ chứa Backend FastAPI, cơ sở dữ liệu SQLite, các service nghiệp vụ, module camera và pipeline AI. Thư mục app_flutter/ chứa ứng dụng Flutter dùng để thao tác với hệ thống. Thư mục notebooks/ chứa các notebook thử nghiệm, tiền xử lý dữ liệu, đánh giá mô hình và so sánh kết quả. Thư mục docs/ chứa báo cáo và tài liệu liên quan. Các thư mục thử nghiệm cũ hoặc beta được giữ lại để tham khảo nhưng không phải là mã nguồn vận hành chính của sản phẩm.

Cách tổ chức này giúp hệ thống không phụ thuộc trực tiếp vào notebook khi chạy sản phẩm. Pipeline AI sau khi thử nghiệm được chuyển thành các module Python trong Backend để có thể tái sử dụng trong live webcam và uploaded video. Backend, Flutter và AI được kiểm thử độc lập, sau đó tích hợp thông qua API.



Hình 4.1. Kiến trúc triển khai của mã nguồn sản phẩm

Lớp	Thành phần chính	Trách nhiệm
Flutter	Dashboard, Members, Register, Surveillance, Logs	Hiển thị giao diện, gửi yêu cầu tới Backend, hiển thị trạng thái, frame, kết quả nhận dạng, nhật ký và cảnh

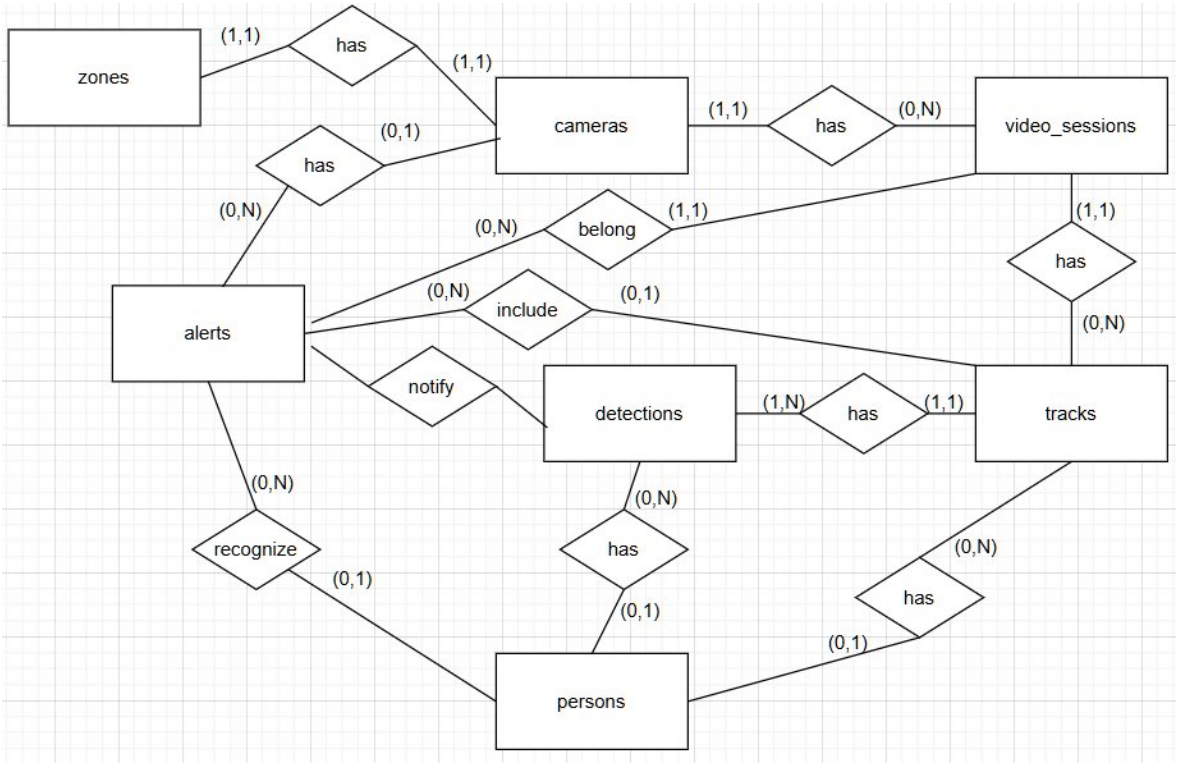
Lớp	Thành phần chính	Trách nhiệm
		báo.
API	FastAPI routes và Pydantic schemas	Định nghĩa endpoint, kiểm tra dữ liệu vào/ra, trả lỗi HTTP nhất quán và giữ hợp đồng giao tiếp với Flutter.
Dịch vụ	MemberService, CameraService, LiveSurveillanceManager, VideoAnalysisManager, LogService, AlertService, ZoneService	Điều phối nghiệp vụ, quản lý vòng đời xử lý, quyết định dữ liệu nào được lưu bền vững hoặc giữ tạm thời.
AI	YOLOv8, ByteTrack, InsightFace, gallery embedding, recognition buffer	Phát hiện người, theo dõi đối tượng, phát hiện khuôn mặt, trích xuất embedding và phân loại known/unknown/low_quality.
Dữ liệu	SQLite và thư mục runtime	Lưu thành viên, embedding, camera, phiên webcam, logs, alerts, zones, snapshot và upload tạm thời.

Bảng 4.1 Các lớp chức năng của hệ thống

4.2 Thiết kế cơ sở dữ liệu quan hệ

Cơ sở dữ liệu của hệ thống sử dụng SQLite, phù hợp với phạm vi sản phẩm thử nghiệm và trình diễn cục bộ. SQLite giúp triển khai đơn giản, không cần máy chủ cơ sở dữ liệu riêng, đồng thời vẫn hỗ trợ khóa ngoại, chỉ mục và truy vấn phục vụ các chức năng chính.

Schema hiện tại gồm các bảng chính: people, face_embeddings, cameras, surveillance_sessions, detection_logs, alerts và zones. Các bảng này phục vụ đầy đủ cho đăng ký khuôn mặt, quản lý camera, giám sát webcam, ghi nhật ký, tạo cảnh báo và cấu hình vùng cấm cơ bản.



Hình 4.2 Mô hình CSDL quan hệ

Chi tiết các bảng chính:

1. people: lưu thông tin thành viên đã đăng ký, gồm mã định danh, tên và thời gian tạo.

2. face_embeddings: lưu thông tin ảnh khuôn mặt và embedding của từng thành viên. Mỗi bản ghi liên kết với một người trong bảng people, đồng thời lưu đường dẫn ảnh khuôn mặt đã cắt và đường dẫn tệp embedding.

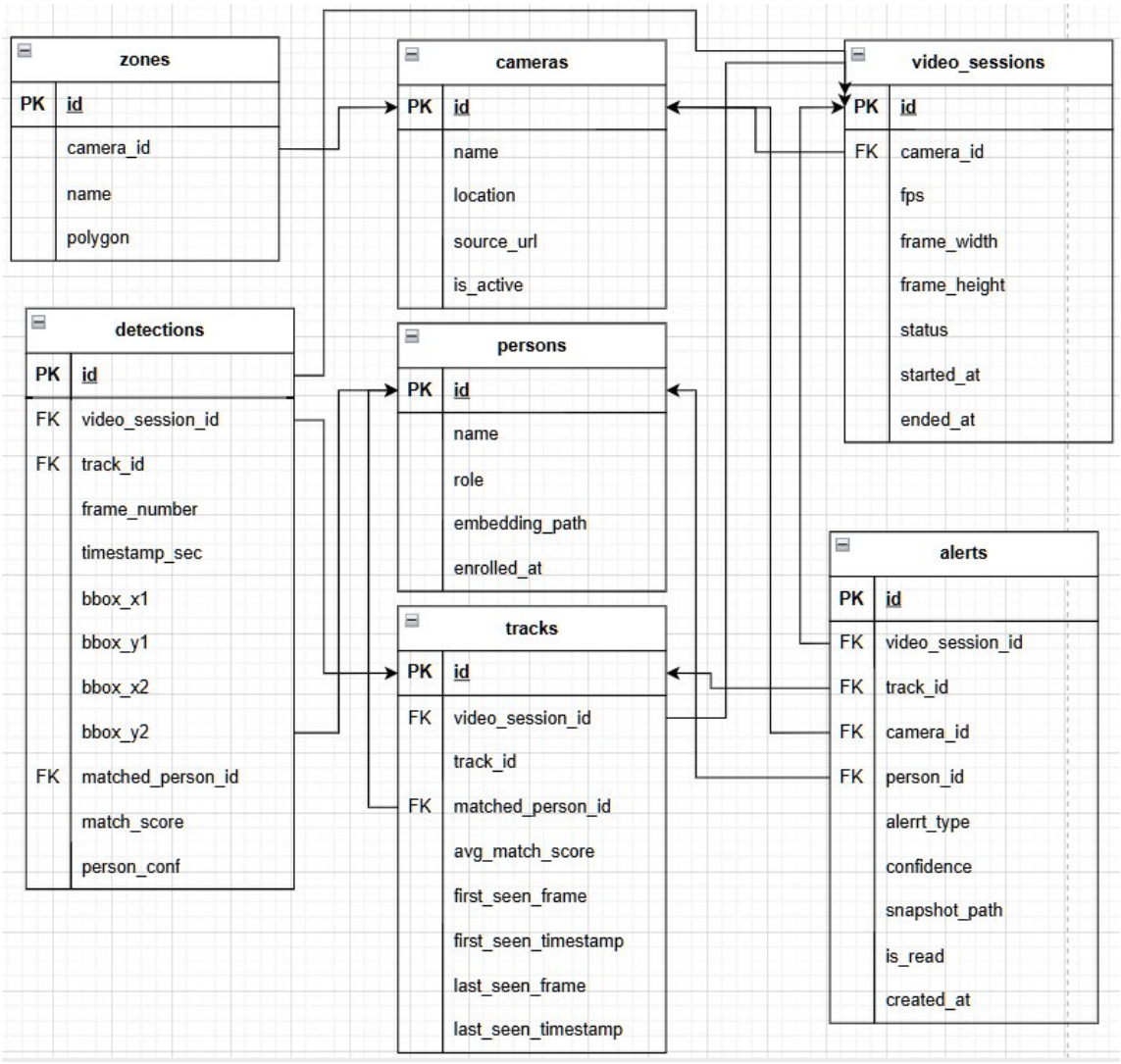
3. cameras: lưu thông tin camera, gồm tên camera, nguồn camera, vị trí và trạng thái hoạt động. Camera có thể được vô hiệu hóa mềm để không dùng cho phiên mới nhưng vẫn giữ liên kết lịch sử.

4. surveillance_sessions: lưu thông tin phiên giám sát trực tiếp bằng webcam, gồm camera, trạng thái phiên, thời gian bắt đầu/kết thúc, FPS trung bình, số frame đã xử lý và thông báo lỗi nếu có.

5. detection_logs: lưu các sự kiện nhận dạng có ý nghĩa trong chế độ live webcam, gồm session, camera, member_id nếu có, track_id, trạng thái known/unknown/low_quality, độ tin cậy, snapshot và thời điểm phát hiện.

6. alerts: lưu cảnh báo sinh ra trong live webcam, gồm loại cảnh báo, nội dung cảnh báo, camera, session, detection log liên quan, snapshot, độ tin cậy và trạng thái đã đọc/chưa đọc.

7. zones: lưu vùng cấm theo camera. Mỗi vùng gồm tên, camera liên quan, danh sách tọa độ điểm dưới dạng JSON và trạng thái hoạt động.



Hình 4.3 Chi tiết thuộc tính, khóa chính, khóa ngoại của cơ sở dữ liệu

Thiết kế này có ranh giới rõ ràng giữa dữ liệu bền vững và dữ liệu tạm thời. Chế độ live webcam lưu phiên, nhật ký, cảnh báo và snapshot. Chế độ uploaded video không tạo bảng job, không tạo phiên giám sát, không ghi detection log, không tạo alert bền vững và không lưu snapshot vào lịch sử.

4.3 Ổn định Backend và quản lý cấu hình

Backend sử dụng một entry point duy nhất tại backend.main. Khi ứng dụng khởi động, FastAPI lifespan tạo các thư mục runtime cần thiết, khởi tạo schema SQLite và chuẩn bị trạng thái hệ thống. Khi ứng dụng dừng, các worker xử lý live webcam hoặc uploaded video được yêu cầu giải phóng tài nguyên.

Các mô hình AI được khởi tạo theo cơ chế lazy loading. Điều này có nghĩa là các API chỉ đọc như health, readiness, danh sách thành viên, logs hoặc alerts không tải YOLOv8 và InsightFace nếu chưa cần thiết. Mô hình chỉ được tải khi có yêu cầu thực sự cần xử lý AI, ví dụ đăng ký khuôn mặt, chạy live webcam hoặc phân tích video tải lên.

Các đường dẫn cơ sở dữ liệu, ảnh khuôn mặt, embedding, snapshot, model và tệp video tạm thời được tập trung trong file cấu hình. Đường dẫn tương đối được giải quyết từ thư mục Backend, giúp hệ thống hoạt động ổn định hơn khi chạy từ script, Docker hoặc môi trường phát triển khác nhau.

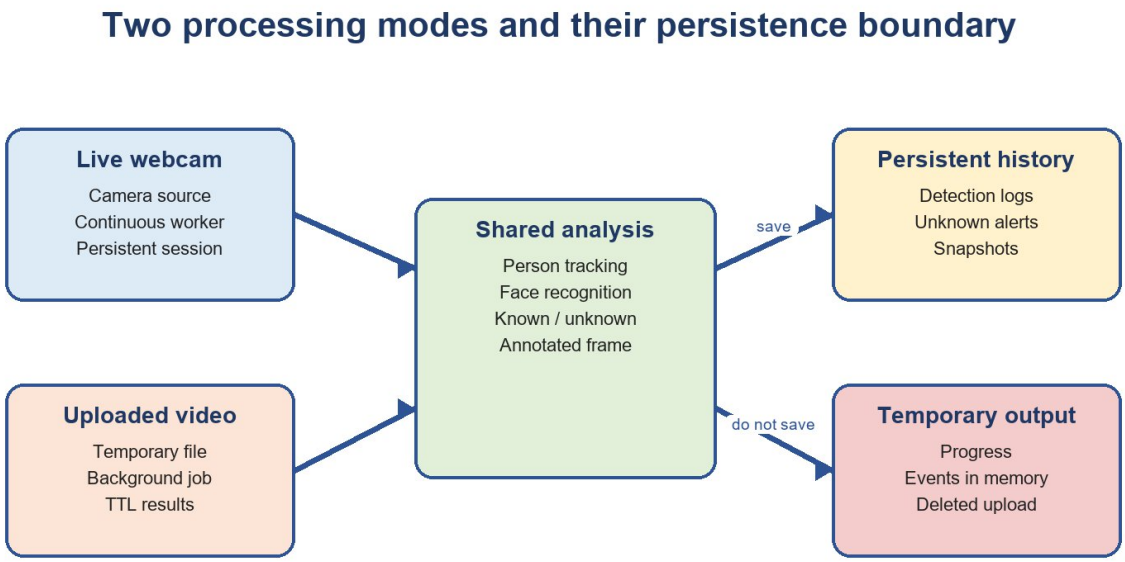
Một số tham số quan trọng có thể cấu hình bằng biến môi trường, gồm ngưỡng nhận dạng khuôn mặt, ngưỡng phát hiện người, số frame xác nhận người lạ, thời gian cooldown cảnh báo, thời gian dwell trong vùng cấm, kích thước video tối đa và thời gian tồn tại của kết quả video tạm thời.

4.4 Cơ sở dữ liệu và ranh giới lưu trữ

Schema SQLite phiên bản hiện tại gồm people, face_embeddings, cameras, surveillance_sessions, detection_logs, alerts và zones. Khóa ngoại và chỉ mục được thiết lập để giữ tính toàn vẹn dữ liệu, hỗ trợ truy vấn theo camera, phiên giám sát, trạng thái cảnh báo và thời gian phát hiện.

Khi xóa thành viên, ảnh khuôn mặt và embedding liên quan được xóa khỏi hệ thống, nhưng lịch sử giám sát đã ghi nhận vẫn được giữ lại. Nếu một bản ghi log hoặc alert từng liên kết với thành viên đó, liên kết định danh có thể được đặt về NULL để bảo toàn lịch sử sự kiện. Khi xóa phiên giám sát, các log và alert thuộc phiên đó được xóa theo quan hệ cascade.

Dữ liệu runtime được chia thành hai nhóm. Nhóm dữ liệu bền vững gồm thông tin thành viên, camera, phiên webcam, log, alert, zone và snapshot cảnh báo. Nhóm dữ liệu tạm thời gồm video tải lên, frame mới nhất của video, tiến độ xử lý và sự kiện tạm thời. Dữ liệu tạm thời không được ghi vào lịch sử giám sát.



Hình 4.4 Ranh giới lưu trữ giữa webcam và video tải lên

Dữ liệu	Webcam trực tiếp	Video tải lên
Phiên xử lý	Lưu vào surveillance_sessions	Không có bảng job hoặc session
Kết quả nhận dạng	Lưu sự kiện có ý nghĩa vào detection_logs	Giữ tạm trong RAM
Cảnh báo người lạ	Lưu vào alerts sau khi xác nhận qua nhiều frame	Chỉ hiển thị tạm thời trong kết quả video
Cảnh báo vùng cấm	Lưu restricted_area alert nếu người lạ ở trong zone quá dwell time	Chưa lưu bền vững
Ảnh minh chứng	Lưu snapshot cho alert live webcam	Không lưu snapshot lịch sử
Tệp đầu vào	Nguồn camera, không sao chép thành file xử lý	Tệp upload được xóa sau khi hoàn tất/lỗi/hủy
Thời gian tồn tại	Theo vòng đời SQLite và thư	TTL tạm thời, mặc định 30 phút

Dữ liệu	Webcam trực tiếp	Video tải lên
	mục snapshot	

Bảng 4.2 Lưu trữ dữ liệu

4.5 Bộ máy AI dùng chung

FrameAnalysisEngine là lõi phân tích khung hình dùng chung cho live webcam và uploaded video. Engine này không tự ghi cơ sở dữ liệu, không tự lưu snapshot và không tự quyết định dữ liệu nào là bền vững. Nhiệm vụ của engine là nhận frame đầu vào, chạy pipeline AI và trả về kết quả phân tích có cấu trúc.

Luồng xử lý của engine gồm các bước chính: nhận frame BGR, dùng YOLOv8 kết hợp ByteTrack để phát hiện và theo dõi người, giới hạn bounding box theo kích thước frame, cắt vùng người, dùng InsightFace để phát hiện khuôn mặt và trích xuất embedding, so khớp embedding với gallery trong bộ nhớ, sau đó đưa kết quả qua recognition buffer theo từng track.

Kết quả trả về gồm frame_index, timestamp, kích thước frame và danh sách track. Mỗi track gồm track_id, bounding box, confidence phát hiện người, trạng thái nhận dạng, member_id nếu khớp, similarity và face confidence nếu có.

TrackRecognitionBuffer duy trì một cửa sổ kết quả riêng cho từng track_id. Cơ chế này giúp giảm dao động danh tính giữa các khung hình liên tiếp. Nếu một track đã được nhận dạng là known nhưng khuôn mặt tạm thời bị khuất, hệ thống có thể kế thừa danh tính đã biết thay vì chuyển ngay sang unknown. Ngược lại, nếu không phát hiện được khuôn mặt rõ và chưa có danh tính ổn định trước đó, track được đánh dấu low_quality để tránh tạo cảnh báo sai.

Hàm annotate frame vẽ bounding box, nhãn known/unknown/low_quality và vùng cấm lên frame. Frame chú thích này được trả về cho Flutter trong live webcam và uploaded video, đồng thời được dùng làm snapshot minh chứng khi live webcam tạo alert.

4.6 Quản lý camera và giám sát webcam

API camera hỗ trợ tạo camera, liệt kê camera active, xem chi tiết camera, cập nhật camera, vô hiệu hóa mềm, kiểm tra nguồn camera và lấy ảnh xem trước. Nguồn camera dạng chuỗi số như "0" được chuyển thành chỉ số webcam OpenCV để hỗ trợ laptop webcam. Camera bị vô hiệu hóa sẽ không xuất hiện trong danh sách mặc định nhưng vẫn giữ liên kết với dữ liệu lịch sử.

Khi test camera hoặc lấy snapshot preview, Backend chỉ mở nguồn camera trong thời gian ngắn để đọc frame, sau đó release tài nguyên. Preview không được lưu vào SQLite hoặc thư mục snapshot. Nếu camera đang được worker live surveillance sử dụng, các thao tác test hoặc preview sẽ bị từ chối để tránh tranh chấp tài nguyên camera.

Khi bắt đầu giám sát, Backend kiểm tra camera tồn tại và đang active, tạo bản ghi trong surveillance_sessions, nạp gallery embedding của thành viên đã đăng ký và khởi động worker nền. Worker đọc frame liên tục, đưa frame vào FrameAnalysisEngine, tạo frame chú thích và cập nhật trạng thái mới nhất trong bộ nhớ để Flutter polling.

Trong quá trình chạy, worker cập nhật số frame đã xử lý, FPS, kết quả nhận dạng mới nhất và JPEG frame chú thích. Khi người dùng dừng giám sát hoặc khi có lỗi, Backend release camera và cập nhật phiên thành stopped hoặc failed, kèm số frame, FPS trung bình và thông báo lỗi nếu có.

4.7 Nhật ký và cảnh báo webcam

Live webcam không ghi một bản ghi cơ sở dữ liệu cho mọi frame. Thay vào đó, hệ thống chỉ lưu các sự kiện có ý nghĩa để giảm số lượng dữ liệu và tránh tạo log dư thừa.

Với người đã biết, hệ thống ghi log khi track lần đầu được nhận dạng hoặc khi danh tính của track thay đổi. Với người lạ, track phải duy trì trạng thái unknown qua số frame xác nhận được cấu hình trước khi hệ thống tạo detection log và alert. Cơ chế này giúp hạn chế cảnh báo sai từ các frame đơn lẻ hoặc kết quả nhận dạng chưa ổn định.

Hệ thống cũng có cơ chế cooldown để tránh tạo nhiều cảnh báo liên tiếp cho cùng một track trong thời gian ngắn. Các frame low_quality hoặc không phát hiện được khuôn mặt rõ không làm tăng unknown streak, nhờ đó tránh việc người quay lưng hoặc khuôn mặt bị che khuất bị kết luận ngay là người lạ.

Đối với vùng cấm, hệ thống kiểm tra bounding box của unknown track với các zone đang active. Nếu người lạ đã được xác nhận nằm trong vùng cấm quá thời gian dwell cấu hình, Backend tạo alert loại restricted_area. Alert này được lưu cùng snapshot frame chú thích để người dùng có bằng chứng khi xem lại.

Snapshot cảnh báo được lưu trong thư mục snapshots. Nếu quá trình ghi cơ sở dữ liệu thất bại, snapshot vừa tạo sẽ được xóa để tránh dữ liệu mồ côi. API logs hỗ trợ xem danh sách và latest log; API alerts hỗ trợ xem danh sách, latest alert, chi tiết alert, snapshot, đánh dấu đã đọc/chưa đọc và xóa alert.

4.8 Phân tích video tải lên tạm thời

VideoAnalysisManager xử lý chế độ uploaded video. Người dùng chọn tệp video từ Flutter, Backend kiểm tra định dạng và kích thước, lưu tạm tệp vào thư mục upload tạm thời và tạo một job xử lý nền.

Trong quá trình xử lý, job có các trạng thái queued, running, completed, failed hoặc cancelled. Backend cập nhật số frame đã xử lý, tổng số frame nếu đọc được, progress, FPS xử lý, frame chú thích mới nhất và danh sách sự kiện tạm thời. Flutter polling các API video-analysis để hiển thị tiến độ và kết quả.

Chế độ uploaded video sử dụng cùng FrameAnalysisEngine với live webcam nhưng không gọi LogService, AlertService hoặc ghi dữ liệu vào SQLite. Các sự kiện known/unknown được thu thập bởi TemporaryEventCollector và chỉ giữ trong bộ nhớ. Kết quả video được đánh dấu non-persistent để phân biệt với lịch sử webcam.

Tệp video upload được xóa trong mọi trường hợp kết thúc: hoàn tất, lỗi hoặc hủy job. Kết quả tạm thời có TTL cấu hình, mặc định 30 phút. Chỉ một job video được phép hoạt động tại một thời điểm, và uploaded video loại trừ lẫn nhau với live surveillance để tránh tranh chấp tài nguyên CPU/GPU trong môi trường trình diễn.

4.9 Tích hợp ứng dụng Flutter

Ứng dụng Flutter sử dụng các model để ánh xạ response từ Backend và service HTTP để tập trung xử lý request, multipart upload, lỗi API và tải ảnh JPEG. Cách tổ chức này giúp giao diện tách biệt khỏi chi tiết endpoint và dễ bảo trì khi Backend thay đổi.

Màn hình Dashboard polling trạng thái Backend, số thành viên, số camera, cảnh báo chưa đọc và detection mới nhất. Màn hình Register cho phép chọn nhiều ảnh và đăng ký thành viên. Màn hình Members hiển thị danh sách thành viên, chi tiết ảnh đã lưu và hỗ trợ xóa thành viên.

Màn hình Surveillance có hai chế độ rõ ràng. Chế độ live webcam cho phép thêm laptop webcam, chọn camera, bắt đầu/dừng giám sát, xem FPS, frame chú thích, danh sách track và cấu hình vùng cấm bằng cách chọn bốn điểm trên frame. Chế độ uploaded video cho phép chọn tệp video, theo dõi tiến độ, xem FPS xử lý, frame chú thích, danh sách sự kiện tạm thời, hủy job hoặc xóa kết quả.

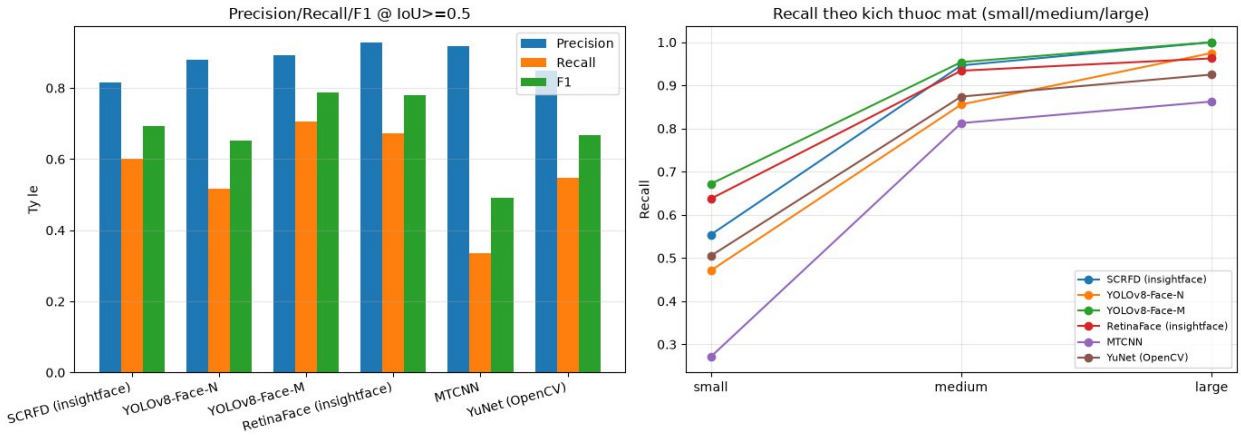
Màn hình Persistent Logs & Alerts chỉ hiển thị dữ liệu bền vững của live webcam. Người dùng có thể xem detection logs, xem alerts, đánh dấu alert đã đọc/chưa đọc, xóa log/alert và truy xuất snapshot minh chứng. Kết quả uploaded video không xuất hiện trong lịch sử này, giúp người dùng hiểu rõ sự khác biệt giữa phân tích tạm thời và giám sát có lưu lịch sử.

CHƯƠNG 5: ĐÁNH GIÁ, SO SÁNH GIỮA CÁC MODELS

5.1 So sánh các model phát hiện khuôn mặt

Trong hệ thống nhận dạng khuôn mặt, bước phát hiện khuôn mặt có vai trò quan trọng vì chất lượng bounding box đầu vào ảnh hưởng trực tiếp đến quá trình cắt khuôn mặt, trích xuất embedding và so khớp danh tính. Nếu mô hình phát hiện khuôn mặt bỏ sót khuôn mặt nhỏ, khuôn mặt nghiêng hoặc khuôn mặt trong điều kiện ánh sáng không ổn định, hệ thống có thể không tạo được embedding hoặc phân loại sai thành low_quality.

Nhóm tiến hành so sánh một số model phát hiện khuôn mặt gồm MTCNN, RetinaFace, SCRFD, YOLOv8-Face-N, YOLOv8-Face-M và YuNet của OpenCV. Các model được đánh giá trên cùng một tập dữ liệu WIDER FACE validation split để bảo đảm điều kiện so sánh thống nhất. Kết quả được chia theo kích thước khuôn mặt gồm small, medium và large, trong đó khuôn mặt small thường khó phát hiện hơn do chiếm ít pixel trong ảnh.



Hình 5.1 Đánh giá models phát hiện người

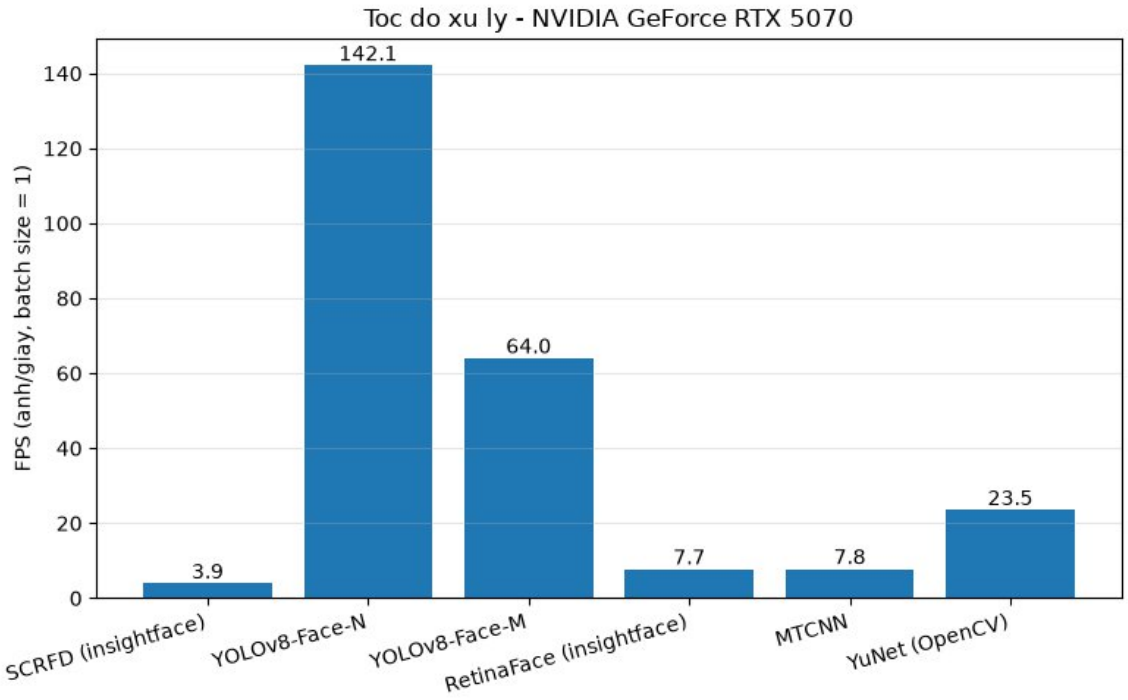
Kết quả recall theo kích thước mặt cho thấy các model học sâu hiện đại có khả năng phát hiện tốt hơn so với một số phương pháp nhẹ truyền thống, đặc biệt ở nhóm khuôn mặt medium và large. Với nhóm khuôn mặt small, hiệu quả phát hiện giảm rõ rệt vì khuôn mặt có ít thông tin đặc trưng hơn, dễ bị ảnh hưởng bởi nhiễu, mờ hoặc góc nhìn.

Model	Nhóm kích thước	Recall	Số khuôn mặt
MTCNN	small	0.271214	5586
MTCNN	medium	0.812596	651
MTCNN	large	0.862500	80
RetinaFace (InsightFace)	small	0.637308	5586

Model	Nhóm kích thước	Recall	Số khuôn mặt
RetinaFace (InsightFace)	medium	0.933948	651
RetinaFace (InsightFace)	large	0.962500	80
SCRFD (InsightFace)	small	0.554064	5586
SCRFD (InsightFace)	medium	0.946237	651
SCRFD (InsightFace)	large	1.000000	80
YOLOv8-Face-M	small	0.671679	5586
YOLOv8-Face-M	medium	0.953917	651
YOLOv8-Face-M	large	1.000000	80
YOLOv8-Face-N	small	0.470999	5586
YOLOv8-Face-N	medium	0.855607	651
YOLOv8-Face-N	large	0.975000	80
YuNet (OpenCV)	small	0.505013	5586
YuNet (OpenCV)	medium	0.874040	651
YuNet (OpenCV)	large	0.925000	80

Từ kết quả trên, YOLOv8-Face-M đạt recall tốt ở cả ba nhóm kích thước, đặc biệt ở nhóm small và medium. SCRFD và RetinaFace cũng cho kết quả ổn định, phù hợp với bài toán nhận dạng khuôn mặt. Trong sản phẩm hiện tại, hệ thống sử dụng InsightFace buffalo_l, trong đó thành phần phát hiện khuôn mặt dựa trên SCRFD và thành phần trích xuất đặc trưng dựa trên ArcFace. Cách lựa chọn này giúp giảm độ phức tạp khi tích hợp vì cùng một thư viện có thể đảm nhiệm cả phát hiện khuôn mặt và trích xuất embedding.

Bên cạnh độ chính xác phát hiện, tốc độ xử lý cũng là yếu tố quan trọng đối với hệ thống giám sát thời gian thực. Một model có recall cao nhưng tốc độ quá thấp có thể làm tăng độ trễ và giảm khả năng xử lý video liên tục.



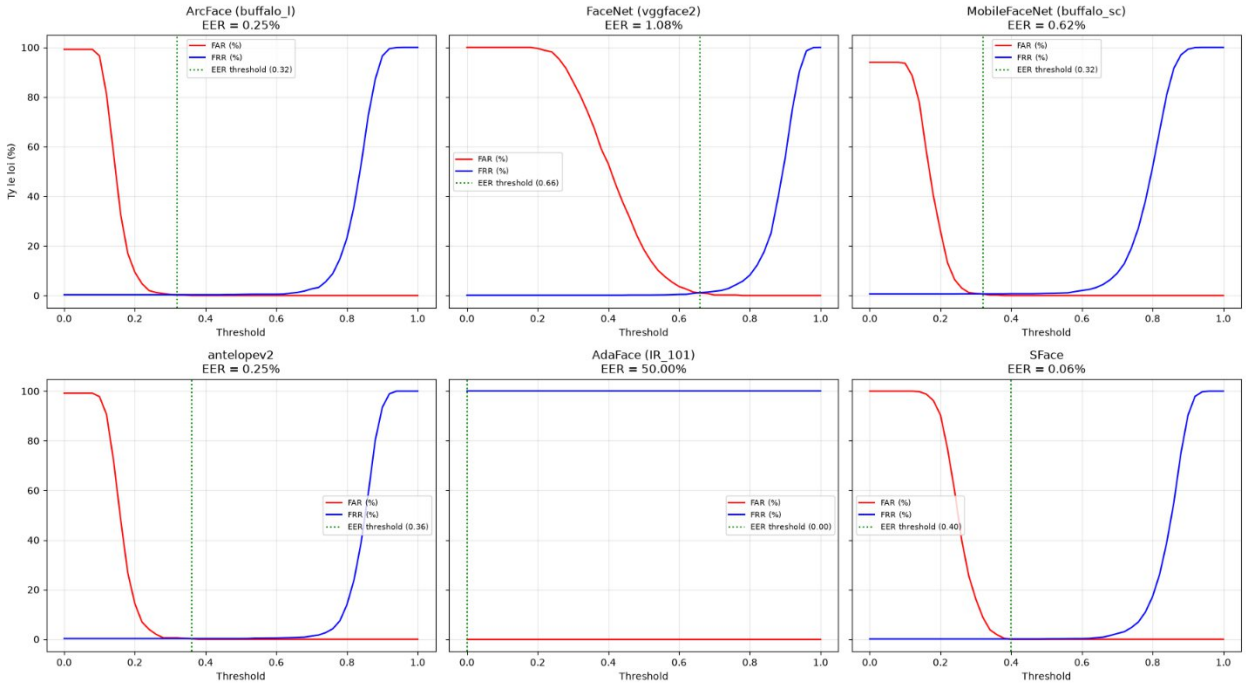
Hình 5. 2 Đánh giá đo tốc độ xử lí của models phát hiện người

5.2 So sánh các model nhận diện khuôn mặt

Sau bước phát hiện khuôn mặt, hệ thống cần trích xuất embedding để biểu diễn khuôn mặt dưới dạng vector đặc trưng. Embedding này được so khớp với gallery đã đăng ký bằng cosine similarity để xác định danh tính hoặc phân loại là unknown.

Nhóm tiến hành so sánh các model trích xuất embedding khuôn mặt như ArcFace, FaceNet, MobileFaceNet, AntelopeV2, AdaFace và SFace. Việc so sánh tập trung vào khả năng phân biệt người quen và người lạ thông qua các chỉ số như FAR, FRR và threshold nhận dạng.

FAR (False Acceptance Rate) thể hiện tỷ lệ người lạ bị nhận nhầm thành người đã đăng ký. FRR (False Rejection Rate) thể hiện tỷ lệ người đã đăng ký bị từ chối hoặc bị phân loại nhầm thành người lạ. Khi threshold tăng, FAR thường giảm vì hệ thống nghiêm ngặt hơn trong việc chấp nhận kết quả khớp. Tuy nhiên, FRR có thể tăng vì người hợp lệ dễ bị từ chối hơn.

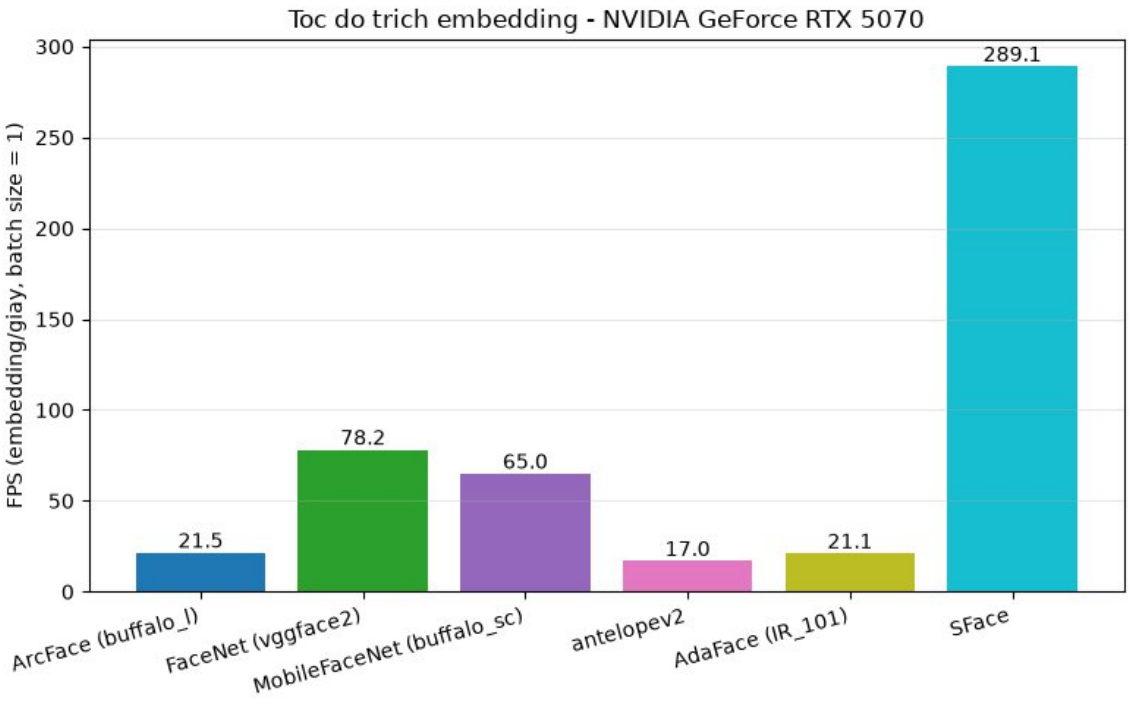


Hình 5.3 Đo FAR, FRR theo thresholds

Dựa trên biểu đồ threshold, nhóm lựa chọn ngưỡng nhận dạng phù hợp để cân bằng giữa an toàn và khả năng sử dụng. Với threshold cố định của hệ thống hiện tại là 0.35, ArcFace cho kết quả phù hợp trong nhóm model được so sánh, với FAR và FRR thấp hơn so với các lựa chọn còn lại trong điều kiện thử nghiệm của nhóm.

Trong sản phẩm hiện tại, nhóm không huấn luyện model nhận dạng từ đầu mà sử dụng model pretrained thông qua InsightFace buffalo_l. Cách tiếp cận này giúp rút ngắn thời gian phát triển, tận dụng mô hình đã được huấn luyện trên dữ liệu lớn và phù hợp với mục tiêu xây dựng sản phẩm thử nghiệm.

Ngoài độ chính xác, thời gian trích xuất embedding cũng ảnh hưởng đến hiệu năng toàn hệ thống. Nếu thời gian tạo embedding quá lâu, tốc độ xử lý webcam hoặc video tải lên sẽ giảm, đặc biệt khi trong khung hình có nhiều người.



Hình 5.4 Đánh giá đo tốc độ trích xuất embedding

Kết quả đo tốc độ trích xuất embedding cho phép nhóm đánh giá chi phí xử lý của từng model. Trong hệ thống thực tế, tốc độ nhận dạng không chỉ phụ thuộc vào model embedding mà còn phụ thuộc vào số lượng khuôn mặt, chất lượng frame, tốc độ phát hiện người, tốc độ tracking và tần suất cần nhận dạng lại theo từng track.

5.3 So sánh các models theo vết người

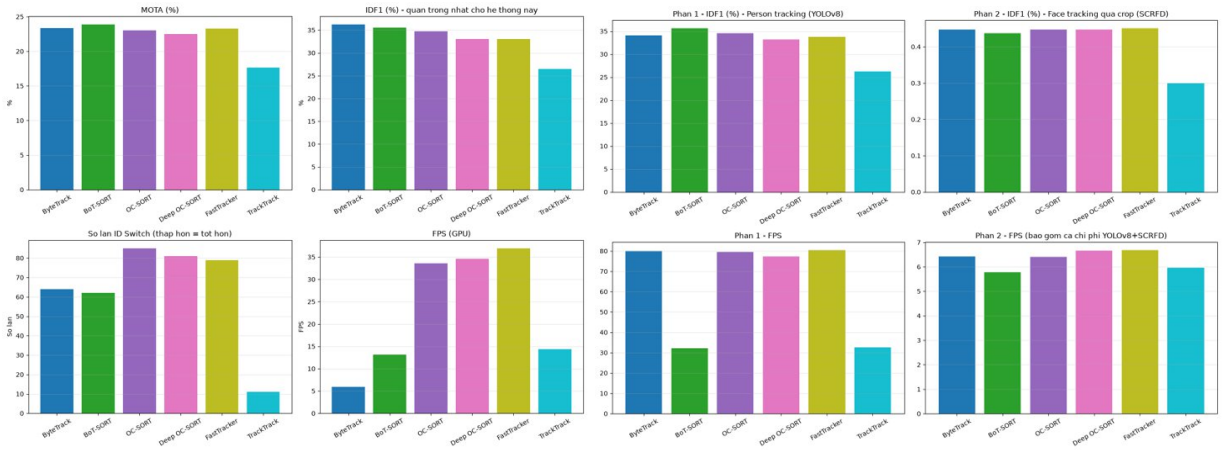
Theo dõi đối tượng là thành phần cần thiết trong hệ thống giám sát video. Nếu chỉ phát hiện người ở từng frame riêng lẻ, hệ thống khó biết một người ở frame hiện tại có phải là cùng người ở frame trước hay không. Vì vậy, tracker được sử dụng để gán và duy trì track_id cho mỗi người qua nhiều khung hình liên tiếp.

Nhóm tiến hành so sánh các tracker như ByteTrack, BoT-SORT, OC-SORT, Deep OC-SORT, FastTracker và TrackTrack trên dữ liệu MOT17. Các tracker được đánh giá sau bước phát hiện người, nhằm so sánh khả năng duy trì định danh đối tượng, hạn chế mất track và hạn chế ID switch.

Tracker	Đặc điểm chính
ByteTrack	Nhanh, đơn giản, phù hợp baseline thời gian thực, không dùng ReID phức tạp.
BoT-SORT	Có thể cải thiện tracking trong một số cảnh phức tạp nhờ bổ sung cơ chế ổn định và tùy chọn ReID.

Tracker	Đặc điểm chính
OC-SORT	Phù hợp với chuyển động phi tuyến tính, tối ưu liên kết dựa trên quan sát chuyển động.
Deep OC-SORT	Bổ sung đặc trưng diện mạo để hỗ trợ theo dõi trong cảnh đông hoặc có che khuất.
FastTracker	Hướng đến tốc độ xử lý nhanh và giảm chi phí tính toán.
TrackTrack	Kết hợp nhiều tín hiệu liên kết, có thể hữu ích trong một số tình huống tracking phức tạp.

Bảng 5.1 Bảng các model trackers của ultralytics



Hình 5.5 Đánh giá các chỉ số cho các model trackers

* Kết quả so sánh tracker phần theo dõi người:

Tracker	MOTA	IDF1	ID Switches	FPS	Composite Score
BoT-SORT	0.240	0.356	63	32.17	0.858
ByteTrack	0.235	0.340	79	80.04	0.714
OC-SORT	0.231	0.346	83	79.45	0.703
FastTracker	0.235	0.338	84	80.40	0.688
Deep OC-SORT	0.225	0.332	71	77.32	0.638
TrackTrack	0.175	0.263	12	32.51	0.200

Theo kết quả trên, BoT-SORT có composite score cao nhất trong phần person tracking. Tuy nhiên, ByteTrack có tốc độ FPS cao hơn đáng kể và độ chính xác tracking vẫn ở mức

gần với nhóm tốt nhất. Vì vậy, trong sản phẩm hiện tại, nhóm lựa chọn ByteTrack để cân bằng giữa tốc độ, độ đơn giản khi tích hợp và khả năng xử lý thời gian thực.

* Kết quả so sánh tracker phần face tracking qua crop:

Tracker	MOTA	IDF1	ID Switches	FPS	Composite Score
TrackTrack	-0.075	0.003	0	5.96	0.550
FastTracker	-0.142	0.005	3	6.68	0.537
ByteTrack	-0.149	0.004	3	6.42	0.496
Deep OC-SORT	-0.149	0.004	3	6.66	0.492
OC-SORT	-0.149	0.004	3	6.40	0.492
BoT-SORT	-0.150	0.004	4	5.77	0.413

Kết quả face tracking qua crop chỉ mang tính tham khảo vì ground truth xấp xỉ và bài toán khó hơn so với tracking toàn thân. Trong sản phẩm hiện tại, nhóm không chọn hướng tracking trực tiếp trên khuôn mặt qua crop làm luồng chính. Thay vào đó, hệ thống phát hiện và theo dõi người bằng YOLOv8 + ByteTrack, sau đó nhận dạng khuôn mặt bên trong vùng người. Cách tiếp cận này phù hợp hơn với giám sát an ninh vì vẫn duy trì được track ngay cả khi khuôn mặt tạm thời bị khuất, quay lưng hoặc quá nhỏ.

Tóm lại, kết quả so sánh cho thấy không có model nào tốt nhất tuyệt đối trong mọi tiêu chí. YOLOv8-Face-M có recall tốt trong bài toán phát hiện khuôn mặt, ArcFace phù hợp cho nhận dạng dựa trên embedding, BoT-SORT có điểm tổng hợp cao trong thử nghiệm tracking, còn ByteTrack có tốc độ cao và dễ tích hợp vào pipeline thời gian thực. Vì mục tiêu của sản phẩm là xây dựng hệ thống thử nghiệm hoạt động end-to-end, nhóm lựa chọn pipeline thực tế gồm YOLOv8 để phát hiện người, ByteTrack để theo dõi đối tượng và InsightFace/ArcFace để phát hiện - nhận dạng khuôn mặt.

CHƯƠNG 6: KẾT QUẢ TRIỂN KHAI VÀ KIỂM THỬ

6.1 Chiến lược kiểm thử

Kiểm thử hệ thống được tổ chức theo nhiều lớp để bảo đảm từng thành phần hoạt động đúng trước khi tích hợp thành sản phẩm hoàn chỉnh. Ở lớp Backend, nhóm sử dụng unit test và API test để kiểm tra hợp đồng dữ liệu, cơ sở dữ liệu, camera, pipeline phân tích, recognition buffer, gallery embedding, live surveillance, video-analysis, logs, alerts, zones và readiness endpoint.

Các API test sử dụng FastAPI TestClient kết hợp với SQLite trong bộ nhớ hoặc môi trường kiểm thử riêng. Các model AI nặng như YOLOv8 và InsightFace được thay bằng test double trong nhiều trường hợp để kiểm thử logic nghiệp vụ mà không cần tải model thật hoặc phụ thuộc vào webcam thật. Cách làm này giúp kiểm thử chạy nhanh, ổn định và có thể phát hiện lỗi ở tầng service, API và schema.

Ở lớp Flutter, nhóm kiểm thử việc ánh xạ dữ liệu JSON từ Backend sang model của ứng dụng, xử lý lỗi API, upload dữ liệu ảnh/video và khả năng hiển thị giao diện chính ở nhiều kích thước màn hình. Các test này giúp bảo đảm Frontend giữ đúng hợp đồng với Backend và không bị lỗi khi dữ liệu trả về thay đổi.

Ngoài kiểm thử tự động, nhóm còn thực hiện kiểm thử thực tế ứng dụng với webcam, video tải lên và dữ liệu thực tế. Việc kiểm thử thực tế tập trung vào luồng thao tác của người dùng, khả năng kết nối Frontend-Backend, hiển thị frame chú thích, trạng thái xử lý, cảnh báo, snapshot, nhật ký và độ ổn định khi vận hành.

Readiness endpoint được sử dụng để kiểm tra nhanh trạng thái hệ thống trước khi chạy ứng dụng. Endpoint này kiểm tra schema cơ sở dữ liệu, khả năng ghi vào các thư mục runtime, trạng thái tài nguyên AI và trạng thái worker mà không tải model nặng. Smoke test được dùng để gọi các endpoint chỉ đọc như health, readiness, members, cameras, surveillance status, logs và alerts nhằm xác nhận Backend có thể khởi động và phục vụ ứng dụng mà không tạo dữ liệu mới.

6.2 Kết quả kiểm thử tự động

Kết quả kiểm thử tự động cho thấy các thành phần chính của hệ thống đã hoạt động ổn định theo hợp đồng phần mềm hiện tại. Backend test bao phủ nhiều nhóm chức năng gồm AI contract, database schema, member API, camera API, surveillance API, video-analysis API, log/alert API, zone API, recognition buffer, event collector và readiness service.

Flutter test kiểm tra các luồng quan trọng ở phía ứng dụng như phân tích response camera/surveillance, marker non-persistent của uploaded video, giữ track_id từ Backend, upload video dạng bytes/stream, chuyển lỗi Backend thành ApiException, gọi endpoint xóa log/alert và render main shell ở desktop, tablet, mobile.

Hạng mục	Kết quả	Bằng chứng
Backend unit/API	108/108 đạt	`python3 -m unittest discover -s backend/tests -v`; bao phủ AI, DB, members, camera, live surveillance, logs, alerts, zones, uploaded video và readiness.
Flutter unit/widget	13/13 đạt	`flutter test`; kiểm tra service contract, upload, error handling, logs/alerts endpoint và render shell ở nhiều kích thước màn hình.
Flutter static analysis	Không có lỗi	`flutter analyze`: No issues found.
OpenAPI	31 paths	Health, members, cameras, surveillance, logs, alerts, video-analysis và zones.
Smoke test	Endpoint chỉ đọc đạt	Kiểm tra health, readiness, members, cameras, surveillance/status, logs và alerts mà không tạo dữ liệu mới.
Kiểm thử thực tế	Đã thực hiện	Kiểm thử ứng dụng bằng webcam, video tải lên và dữ liệu thực tế để kiểm tra luồng vận hành end-to-end.

Bảng 6.1 Kết quả kiểm thử tự động

Kết quả trên cho thấy hệ thống đã có nền tảng kiểm thử khá đầy đủ cho phiên bản sản phẩm thử nghiệm. Tuy nhiên, kiểm thử tự động không thay thế hoàn toàn kiểm thử thực tế với model thật, camera thật và dữ liệu môi trường thật. Vì vậy, nhóm kết hợp cả test tự động, smoke test và kiểm thử thủ công trên ứng dụng.

6.3 Các kịch bản quan trọng đã kiểm chứng

Các kịch bản quan trọng đã được kiểm chứng gồm:

- Camera không tồn tại hoặc bị vô hiệu hóa không thể bắt đầu giám sát.

- Camera được release khi đọc ảnh thành công, thất bại hoặc khi worker kết thúc.

- Không thể chạy live surveillance đồng thời với job video tải lên.

- Uploaded video không thể bắt đầu khi live surveillance đang chạy.

- Người lạ phải được xác nhận qua nhiều khung hình trước khi tạo cảnh báo.

- Cảnh báo người lạ tuân thủ cooldown để tránh tạo nhiều alert trùng lặp cho cùng một track.

- Khung hình low_quality không làm tăng chuỗi xác nhận người lạ.

- Người đã biết được ghi log khi track lần đầu được nhận dạng hoặc khi danh tính thay đổi.

- Snapshot cảnh báo được lưu khi có alert hợp lệ.

- Nếu giao dịch cơ sở dữ liệu thất bại, snapshot vừa tạo được xóa để tránh file mồ côi.

- Job video hoàn thành, lỗi hoặc hủy đều xóa tệp đầu vào tạm.

- Phân tích video tải lên không làm thay đổi surveillance_sessions, detection_logs hoặc alerts.

- Xóa thành viên giữ lịch sử giám sát và xóa liên kết danh tính theo đúng khóa ngoại.

- Zone có thể được tạo, liệt kê, cập nhật, bật/tắt và xóa thông qua API.

- Restricted-area alert được tạo khi unknown track nằm trong vùng cấm quá thời gian dwell cấu hình.

- Readiness phát hiện schema thiếu bảng và trả trạng thái degraded.

- Flutter xử lý được response Backend, lỗi API, upload file và hiển thị giao diện chính.

Các kịch bản trên tập trung vào những rủi ro chính của hệ thống: quản lý tài nguyên camera, ranh giới lưu trữ giữa hai chế độ, tránh cảnh báo sai, tránh dữ liệu mồ côi, bảo đảm API ổn định và bảo đảm giao diện hoạt động đúng với dữ liệu Backend.

6.4 Kết quả chức năng theo hai chế độ

Tiêu chí	Live webcam	Uploaded video
Bắt đầu từ Flutter	Chọn camera hoặc thêm laptop webcam, sau đó nhấn Start.	Chọn tệp video hỗ trợ từ thiết bị.

Tiêu chí	Live webcam	Uploaded video
Trạng thái xử lý	running, stopped, failed; hiển thị FPS, số frame, lỗi nếu có.	queued, running, completed, failed, cancelled; hiển thị progress và FPS xử lý.
Khung hình	JPEG chú thích mới nhất từ webcam	JPEG chú thích mới nhất từ video đang xử lý.
Kết quả nhận dạng	Track nhận dạng hiện tại gồm known, unknown, low_quality.	Danh sách sự kiện tạm thời gồm known/unknown events.
Nhật ký	Có lưu detection_logs cho sự kiện có ý nghĩa.	Không lưu detection_logs.
Cảnh báo	Có lưu unknown_person và restricted_area alerts.	Không lưu alerts bền vững.
Snapshot	Có lưu snapshot cho alert live webcam.	Không lưu snapshot lịch sử.
Vùng cấm	Có hỗ trợ zones và cảnh báo restricted_area.	Không áp dụng lưu cảnh báo vùng cấm bền vững.
Kết thúc	Cập nhật surveillance session và release camera.	Xóa file upload; kết quả tạm thời hết hạn theo TTL.

Bảng 6.2 Kết quả chức năng theo hai chế độ

6.5 Khả năng vận hành và chẩn đoán

Nhóm bổ sung các công cụ vận hành để hỗ trợ chạy thử và kiểm tra hệ thống. Script start_backend.ps1 hỗ trợ kiểm tra dependency và chạy Uvicorn. Script start_flutter.ps1 kiểm tra readiness trước khi mở ứng dụng Flutter. Script smoke_test.py thực hiện kiểm tra các endpoint chỉ đọc. Script verify_project.ps1 chạy toàn bộ kiểm thử Backend, Flutter static analysis và Flutter test.

README ở thư mục gốc được cập nhật thành runbook thống nhất, hướng dẫn cách cài đặt, chạy Backend, chạy Flutter, chạy Docker Compose, kiểm tra readiness, xử lý lỗi thường gặp và phân biệt hai chế độ live webcam với uploaded video.

Endpoint /health/readiness trả về trạng thái database, storage, AI assets và worker. Trạng thái model ở dạng download_on_first_use chỉ mang tính thông tin và không làm Backend degraded, vì hệ thống chủ động lazy load model khi cần. Backend chỉ degraded khi schema không đúng, thiếu bảng cần thiết hoặc thư mục runtime không sẵn sàng ghi.

Khả năng chẩn đoán này giúp nhóm dễ xác định lỗi trong quá trình trình diễn, ví dụ Backend chưa chạy, camera không mở được, model chưa tải, uploaded video đang chiếm pipeline, hoặc thư mục runtime không ghi được. Đây là phần quan trọng để sản phẩm thử nghiệm có thể được chạy lại ổn định trên nhiều môi trường phát triển khác nhau.

CHƯƠNG 7: ĐÁNH GIÁ VÀ KẾ HOẠCH TIẾP THEO

7.1 Mức độ hoàn thành theo phân công

Trong quá trình thực hiện đề tài, nhóm đã hoàn thành phần lớn các chức năng cốt lõi của hệ thống nhận dạng khuôn mặt, giám sát an ninh và phân tích video thời gian thực. Các thành viên trong nhóm được phân công theo từng mảng chính gồm AI, Backend, Frontend, cơ sở dữ liệu, kiểm thử, dữ liệu thực tế và báo cáo.

Phần AI đã xây dựng được pipeline phát hiện người, theo dõi đối tượng và nhận dạng khuôn mặt có thể tích hợp vào Backend. Hệ thống sử dụng YOLOv8 để phát hiện người, ByteTrack để duy trì track_id qua nhiều khung hình và InsightFace/ArcFace để phát hiện khuôn mặt, trích xuất embedding và so khớp với gallery đã đăng ký. Các thử nghiệm notebook và biểu đồ so sánh mô hình do nhóm thực hiện đã hỗ trợ việc lựa chọn pipeline phù hợp cho sản phẩm.

Phần dữ liệu và cơ sở dữ liệu đã hình thành schema SQLite phục vụ các thực thể chính của hệ thống, gồm people, face_embeddings, cameras, surveillance_sessions, detection_logs, alerts và zones. Hệ thống cũng tổ chức thư mục lưu ảnh khuôn mặt, embedding, snapshot cảnh báo, model và video upload tạm thời. Ranh giới lưu trữ giữa live webcam và uploaded video được thiết kế rõ ràng: live webcam lưu lịch sử, còn uploaded video chỉ giữ kết quả tạm thời.

Phần ứng dụng và tích hợp đã chuyển pipeline thử nghiệm thành sản phẩm có thể vận hành thông qua Flutter và FastAPI. Ứng dụng Flutter đã có các màn hình Dashboard, Members, Register, Surveillance và Logs. Backend đã có các API cho quản lý thành viên, camera, giám sát live webcam, phân tích video tải lên, logs, alerts và zones. Hai chế độ xử lý live webcam và uploaded video đã được tích hợp end-to-end.

Phần kiểm thử đã được thực hiện bằng cả kiểm thử tự động và kiểm thử thực tế. Kiểm thử tự động bao phủ Backend, API, schema, camera, surveillance, video-analysis, logs, alerts, zones, recognition buffer và Flutter service/widget. Ngoài ra, nhóm đã kiểm thử thực tế ứng dụng với webcam, video tải lên và dữ liệu thực tế để đánh giá luồng thao tác, frame chú thích, cảnh báo, snapshot, độ ổn định và khả năng sử dụng của hệ thống.

Nhìn chung, hệ thống hiện đã đáp ứng được mục tiêu xây dựng sản phẩm thử nghiệm có khả năng đăng ký khuôn mặt, quản lý thành viên, nhận dạng người quen/người lạ, giám sát webcam, phân tích video tải lên, cấu hình vùng cấm cơ bản, tạo cảnh báo, lưu nhật ký và hiển thị kết quả trên ứng dụng Flutter. Các chức năng nâng cao hơn vẫn được xác định là hướng phát triển tiếp theo.

7.2 Hạn chế hiện tại

Mặc dù hệ thống đã hoàn thành các chức năng cốt lõi, phiên bản hiện tại vẫn còn một số hạn chế.

Thứ nhất, hệ thống đã có kiểm thử tự động và kiểm thử thực tế, nhưng kết quả benchmark chính thức trên một bộ dữ liệu đánh giá chung vẫn cần được hoàn thiện thêm. Các chỉ số như Accuracy, FAR, FRR, Precision, Recall, F1-score, FPS và độ trễ đã được định hướng và có thử nghiệm so sánh mô hình, nhưng cần chuẩn hóa lại trên cùng điều kiện phần cứng, cùng tập dữ liệu và cùng quy trình đo để có kết luận chính thức hơn.

Thứ hai, chức năng vùng cấm đã được triển khai ở mức cơ bản, nhưng vẫn còn giới hạn. Người dùng hiện có thể tạo zone bằng bốn điểm trên giao diện Flutter và hệ thống có thể sinh cảnh báo restricted_area khi người lạ ở trong vùng quá thời gian cấu hình. Tuy nhiên, hệ thống chưa hỗ trợ chỉnh sửa polygon nâng cao, nhiều loại vùng khác nhau hoặc giao diện quản lý rule phức tạp.

Thứ ba, hệ thống mới triển khai các logic cảnh báo chính gồm unknown_person và restricted_area. Các hành vi nâng cao như loitering tổng quát, phát hiện tụ tập đông người, phát hiện chuyển động bất thường hoặc rule engine cấu hình động chưa được triển khai trong phiên bản hiện tại.

Thứ tư, hệ thống chưa có chức năng xác thực tài khoản và phân quyền người dùng. Ứng dụng hiện tập trung vào môi trường trình diễn cục bộ, chưa có đăng nhập, role Admin/Operator, bảo vệ API bằng token hoặc audit log cho thao tác người dùng.

Thứ năm, hệ thống chưa có quản lý thiết bị và push notification. Cảnh báo hiện được xem trực tiếp trên ứng dụng Flutter thông qua màn hình Logs, nhưng chưa gửi thông báo nền đến thiết bị di động qua Firebase Cloud Messaging hoặc cơ chế tương tự.

Thứ sáu, hệ thống hiện giới hạn một pipeline AI nặng tại một thời điểm. Live webcam và uploaded video được thiết kế loại trừ lẫn nhau để tránh tranh chấp tài nguyên CPU/GPU. Cách làm này phù hợp với sản phẩm thử nghiệm, nhưng chưa đáp ứng mục tiêu xử lý đồng thời nhiều camera trong môi trường triển khai lớn.

Thứ bảy, SQLite và lưu trữ cục bộ phù hợp cho phát triển, kiểm thử và trình diễn, nhưng chưa phải kiến trúc tối ưu cho triển khai phân tán hoặc lưu trữ quy mô lớn. Nếu triển khai thực tế, hệ thống cần xem xét cơ sở dữ liệu máy chủ, lưu trữ tập trung, sao lưu dữ liệu và cơ chế bảo vệ dữ liệu khuôn mặt.

7.3 Kế hoạch giai đoạn tiếp theo

Trong giai đoạn tiếp theo, nhóm định hướng tiếp tục hoàn thiện hệ thống theo các nội dung sau:

Hoàn thiện đánh giá định lượng chính thức: xây dựng bộ dữ liệu kiểm thử chung cho gallery và query, chạy lại các mô hình trong cùng điều kiện phần cứng, ghi nhận Accuracy, FAR, FRR, Precision, Recall, F1-score, FPS và độ trễ end-to-end. Kết quả này sẽ giúp đánh giá khách quan hơn chất lượng nhận dạng và hiệu năng của hệ thống.

Mở rộng chức năng vùng cấm và luật hành vi: bổ sung khả năng chỉnh sửa vùng trực quan hơn, hỗ trợ nhiều loại vùng, xây dựng rule engine cấu hình động và cho phép người dùng bật/tắt các luật cảnh báo theo từng camera hoặc từng vùng.

Triển khai loitering và các hành vi nâng cao: sử dụng thông tin track_id, thời gian tồn tại trong vùng và trạng thái di chuyển để phát hiện người xuất hiện quá lâu trong một khu vực. Sau đó có thể mở rộng sang các tình huống như tụ tập đông người hoặc các mẫu hành vi bất thường khác.

Bổ sung xác thực và phân quyền: xây dựng chức năng đăng nhập, quản lý người dùng, phân quyền Admin/Operator, bảo vệ API bằng token và ghi nhận audit log cho các thao tác quan trọng như xóa dữ liệu, thay đổi camera hoặc xử lý cảnh báo.

Bổ sung quản lý thiết bị và push notification: lưu token thiết bị, tích hợp Firebase Cloud Messaging hoặc cơ chế thông báo phù hợp để gửi cảnh báo mới đến thiết bị người dùng theo thời gian thực.

Tối ưu hiệu năng xử lý: đánh giá tốc độ từng thành phần trong pipeline, tối ưu tần suất nhận dạng khuôn mặt, giảm độ trễ hiển thị frame, cải thiện xử lý video có nhiều người và xem xét khả năng chạy trên GPU nếu môi trường triển khai cho phép.

Mở rộng khả năng vận hành: hỗ trợ nhiều camera tốt hơn, cải thiện cấu hình camera IP/RTSP, bổ sung cơ chế giám sát lỗi runtime, sao lưu dữ liệu, quản lý model và đóng gói triển khai ổn định hơn cho môi trường thực tế.

Hoàn thiện tài liệu và sản phẩm trình diễn: bổ sung ảnh minh chứng từ kiểm thử thực tế, cập nhật biểu đồ đánh giá, ghi rõ giới hạn hệ thống và chuẩn hóa hướng dẫn cài đặt, vận hành, kiểm thử để người khác có thể chạy lại sản phẩm dễ dàng.

KẾT LUẬN

Qua quá trình thực hiện đề tài "Xây dựng hệ thống nhận dạng khuôn mặt, giám sát an ninh và phân tích video thời gian thực", nhóm đã xây dựng được một sản phẩm thử nghiệm có kiến trúc rõ ràng và có khả năng vận hành end-to-end. Hệ thống gồm ứng dụng Flutter ở phía người dùng, Backend FastAPI xử lý nghiệp vụ và tích hợp AI, cơ sở dữ liệu SQLite để lưu trữ dữ liệu cục bộ, cùng các module AI phục vụ phát hiện người, theo dõi đối tượng và nhận dạng khuôn mặt.

Về chức năng, hệ thống đã hỗ trợ đăng ký và quản lý thành viên bằng ảnh khuôn mặt, lưu ảnh khuôn mặt và embedding, quản lý camera, giám sát trực tiếp bằng webcam, phân tích video tải lên, hiển thị frame chú thích, nhận dạng người quen/người lạ, ghi nhật ký, tạo cảnh báo và xem lại snapshot minh chứng. Pipeline AI sử dụng YOLOv8 để phát hiện người, ByteTrack để theo dõi đối tượng và InsightFace/ArcFace để phát hiện khuôn mặt, trích xuất embedding và so khớp với gallery đã đăng ký.

Một điểm quan trọng trong thiết kế của hệ thống là phân tách rõ ràng giữa hai chế độ xử lý. Chế độ live webcam phục vụ giám sát trực tiếp nên lưu phiên giám sát, detection logs, alerts và snapshot cảnh báo. Ngược lại, chế độ uploaded video phục vụ phân tích tạm thời nên chỉ giữ tiến độ, frame và sự kiện trong bộ nhớ, không ghi dữ liệu vào lịch sử giám sát và tự xóa tệp sau khi xử lý. Cách thiết kế này giúp dữ liệu được quản lý rõ ràng, tránh lẫn lộn giữa giám sát thực tế và phân tích thử nghiệm.

Hệ thống cũng đã triển khai các cơ chế nhằm tăng độ ổn định và giảm cảnh báo sai, bao gồm xác nhận người lạ qua nhiều khung hình, cooldown để tránh tạo cảnh báo trùng lặp, phân biệt trạng thái low_quality với unknown, cleanup snapshot khi giao dịch cơ sở dữ liệu thất bại và giới hạn chỉ chạy một pipeline AI nặng tại một thời điểm. Ngoài ra, chức năng vùng cấm đã được triển khai ở mức cơ bản, cho phép người dùng chọn bốn điểm trên giao diện Flutter, lưu zone vào cơ sở dữ liệu và tạo cảnh báo restricted_area khi người lạ xuất hiện trong vùng quá thời gian cấu hình.

Về kiểm thử, hệ thống đã đạt 108 kiểm thử Backend và 13 kiểm thử Flutter; Flutter static analysis không phát hiện lỗi. Nhóm cũng đã thực hiện kiểm thử thực tế ứng dụng với webcam, video tải lên và dữ liệu thực tế để kiểm tra luồng vận hành, giao diện, cảnh báo, snapshot và độ ổn định của sản phẩm. Bên cạnh đó, các notebook và biểu đồ so sánh mô hình giúp nhóm đánh giá, lựa chọn và giải thích pipeline AI phù hợp với mục tiêu của đề tài.

Tuy nhiên, hệ thống hiện vẫn còn một số giới hạn. Các chức năng nâng cao như loitering, rule engine cấu hình động, xác thực tài khoản, phân quyền người dùng, quản lý thiết bị, push notification và xử lý đồng thời nhiều camera chưa được triển khai hoàn chỉnh. Ngoài ra, kết quả đánh giá mô hình cần tiếp tục được chuẩn hóa trên bộ dữ liệu chung và cùng

điều kiện phần cứng để có số liệu chính thức hơn về Accuracy, FAR, FRR, Precision, Recall, F1-score, FPS và độ trễ.

Nhìn chung, đề tài đã đạt được mục tiêu xây dựng một hệ thống giám sát an ninh thử nghiệm có khả năng nhận dạng khuôn mặt, phân tích video theo thời gian thực, lưu nhật ký và tạo cảnh báo trên ứng dụng. Đây là nền tảng để nhóm tiếp tục phát triển các chức năng nâng cao, tối ưu hiệu năng và hoàn thiện hệ thống theo hướng gần hơn với một giải pháp giám sát thực tế.

TÀI LIỆU THAM KHẢO

[1] FastAPI Documentation, "FastAPI framework, high performance, easy to learn, fast to code, ready for production."
https://fastapi.tiangolo.com/

[2] Flutter Documentation, "Build apps for any screen."
https://docs.flutter.dev/

[3] SQLite Documentation, "SQLite Documentation."
https://www.sqlite.org/docs.html

[4] OpenCV Documentation, "Open Source Computer Vision Library."
https://docs.opencv.org/

[5] Ultralytics, "YOLOv8 Documentation."
https://docs.ultralytics.com/models/yolov8/

[6] G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," GitHub repository.
https://github.com/ultralytics/ultralytics

[7] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," ECCV, 2022.
https://arxiv.org/abs/2110.06864

[8] J. Deng, J. Guo, N. Xue, S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," CVPR, 2019.
https://arxiv.org/abs/1801.07698

[9] InsightFace Project, "2D and 3D Face Analysis Project."
https://github.com/deepinsight/insightface

[10] J. Guo, J. Deng, A. Lattas, S. Zafeiriou, "Sample and Computation Redistribution for Efficient Face Detection," SCRFD, arXiv, 2021.
https://arxiv.org/abs/2105.04714

[11] S. Yang, P. Luo, C. C. Loy, X. Tang, "WIDER FACE: A Face Detection Benchmark," CVPR, 2016.
http://shuoyang1213.me/WIDERFACE/

[12] A. Milan, L. Leal-Taixe, I. Reid, S. Roth, K. Schindler, "MOT16: A Benchmark for Multi-Object Tracking," arXiv, 2016.
https://motchallenge.net/

[13] Python Software Foundation, “Python Documentation.”
https://docs.python.org/3/

[14] Docker Documentation, “Docker Docs.”
https://docs.docker.com/

[15] Pydantic Documentation, “Data validation using Python type hints.”
https://docs.pydantic.dev/

Link GitHub của nhóm: https://github.com/kaffeinesloth/AI_Surveillance_System